

In a data query, you can use the **GENERATOR** operator to generate a stream of values. You can either generate a range of gener

Tip: This operator is useful when you want to quickly generate a dataset to feed your data queries or data fusions in Data Pe

This page discusses:

- [Input](#)
- [Generate a Range of Numeric or Date Values](#)
- [Enter a Set of Values with a List Expression](#)
- [Output](#)

Input

N/A.

Generate a Range of Numeric or Date Values

The **Range Strategy** generates a list of values between a **Start inclusive** value (it includes the first value) and an **End exclusive** va

You can define a range of:

- Numeric values: Integer, Float, or Decimal values. If the value is numeric, the **Step** is also numeric.
- Dates: Date or LocalDateTime values.

If the value is a date, the **Step** can be:

- A Period, which takes the calendar into account to calculate the step between dates. The goal is to handle relat
- A Duration, which does not take the calendar into account but increments dates with n seconds. The goal of is to

Optionally, you can transform the output values by defining an **Output expression**. This expression has a unique variable called v

Note: If the order of values in the output of the generator has no importance, you can clear the **Keep order** check box for better

Example 1: To generate a range of float values from 1.0 to 4.0, separated with a step of 0.5, you can define:

- **Start inclusive:** 1.0
- **End exclusive:** 4.0
- **Step:** 0.5

In the output, you get 6 float values: 1.0, 1.5, 2.0, 2.5, 3.0, and 3.5.

Example 2: To calculate and return the cosinus for each value of a range, you can define `Math.cos(value)` as **Output expression**.

For the parameters defined in **Example 1**, the output returns: 0.5403023058681398, 0.0707372016677029, -0.41614683654714

Example 3: To generate a range of dates with a step of 7 days, between March 26, 2024 and April 26, 2024, you can define:

- **Start inclusive:** `Date.of(year: 2024, month: 3 , day: 26 )`
- **End exclusive:** `Date.of(year: 2024, month: 4 , day: 26)`

- **Step:** Period.of(days:7)

In the output, you get the following date values: 2024-03-26, 2024-04-02, 2024-04-09, 2024-04-16, and 2024-04-23.

Example 4: To generate a range of LocalDateTimes with a step of 4 hours, between April 2, 2024 18:30:00 and April 3, 2024 07:15:00

- **Start inclusive:** LocalDateTime.of(2024, 4, 2, 18, 30, 0)
- **End exclusive:** LocalDateTime.of(2024, 4, 3, 7, 15, 0)
- **Step:** Duration.of(hours: 4)

In the output, you get the following values: 2024-04-02 18:30:00, 2024-04-02 22:30:00, 2024-04-03 02:30:00, and 2024-04-03 06:30:00

Enter a Set of Values with a List Expression

The **Iterate over List Strategy** creates a flow using a list defined in the **List expression**.

For example, the following **List expression** returns 3 fields country, team, and teamcount.

Important: Surround string values with "" double quotes and numeric values without quotes.

```
[{country:"France",team:"Development",teamCount:60},  
{country:"United Kingdom",team:"Sales",teamCount:5},  
{country:"Germany",team:"Development",teamCount:20},  
{country:"USA",team:"Marketing",teamCount:20},  
{country:"India",team:"Development",teamCount:80},  
{country:"Japan",team:"Sales",teamCount:5},  
{country:"China",team:"Sales",teamCount:6},  
{country:"Australia",team:"Sales",teamCount:4}]
```

The following **Table** visualization shows the data generated out of the **List expression** defined above (after the **GENERATOR** has been executed)

country	team	teamCount
Australia	Sales	4
China	Sales	6
France	Development	60
Germany	Development	20
India	Development	80
Japan	Sales	5
USA	Marketing	20
United Kingdom	Sales	5

Output

A stream containing n values in output.

 SCHEMA QUERY

In a data query, you can use the **SCHEMA QUERY** operator to introspect a Data Model.

This page discusses:

- [Configuration](#)
- [Output](#)
- [Element Types](#)
- [Example](#)

Configuration

Configure a **Clause** with the following parameters:

Parameter	Description
Variable	The name of the variable.
Introspected schema	The schema you want to introspect.
Element type	The element type that you want to stream. For example, a Class Definition , an Attribute D For more information, see Element Types .
Predicate	A predicate to filter returned elements.

Output

This operator has one output that streams multiple elements. The element type depends on the configurati

Element Types

The **SCHEMA QUERY** operator can stream different element types.

Type	Description
Class Definition	Represents the definition of a class. For example: <pre>class Person { name: String</pre>

	<pre> age: Integer } </pre> With this element type, the operator streams all class definitions in your Data Model.
Attribute Definition	Represents the definition of a reusable attribute. For example: <pre>attribute Age : Integer { @Positive } </pre> With this element type, the operator streams all attribute definitions in your Data Model.
Attribute	Represents a class attribute. For example: <pre>class Person { name: String // <-- a class attribute age: Integer // <-- another class attribute } </pre> With this element type, the operator streams all annotation definitions in your Data Model.
Class definition Annotation and Attribute Annotation	An annotation represents an annotation instance linked to a class or an attribute. For example: <pre>class Person { @AnAnnotation // <-- annotation instance linked to a class definition name: String age: Integer { @DefaultValue(value: 12) // <-- annotation instance linked to an attribute } } </pre> If you choose <code>class definition annotation</code> or <code>attribute annotation</code>, the operator returns the <code>Item<schema.Annotation></code>.

Example

To get all class definitions with a numerical attribute, you need two operators:

- A **SCHEMA QUERY** operator with the **Attribute** element type and the following **Predicate**:

```
attribute->parentAttributes.anyMatch((attr) => attr.fqName == "core.Integer")
```

- A **MAP** operator with the following **Output** expression: `attribute.declarationClass`

In a data query, you can use the **SEARCH** operator to target a specific item type in the index, including its type subclasses.

For example, if you search for the `employee` item type, you get entries for `engineers`, `consultants`, `managers`, etc., which are subclasses.

This page discusses:

- [Input](#)
- [Output](#)
- [Standard Configuration](#)
- [Matching Annotations and Text Scoring](#)
- [Builtin Matching Annotation](#)
- [Other Use of Matching Annotations](#)

Input

The **SEARCH** operator is a source so it has no input.

Output

By default, the output of this operator is a stream with multiple elements. The type of the elements in the stream is `Item`.

If you select the **Advanced > Use matching annotations** option, the type of element in the output stream is `MatchedItem`.

Note: The **SEARCH** operator does not order the output stream, even if you activate the text scoring in **Advanced > Use matching annotations**.

Standard Configuration

The operator returns the selected index unit elements that match a predicate (Boolean condition).

1. Define an **Index unit**.

Note: You can only select the index units created on the same tenant, and shared with users having the **SEARCH** operator.

- In **Value** mode, you can select an index unit directly. The default index unit is `3DSpace`.
 - In **\$ (parameter)** mode, you can select an **Index Unit Parameter** defined beforehand in **\$ Index Unit Parameter**.
2. **Optional:** To explore the Index Unit and find the class to use in **Variable**, click  to open the **Index Unit Explorer**.
 3. In **Variable**, select an Index Unit class, and specify the name of the variable, that you will use in the **Expression**.

For example, you select the `Person` type and enter `person` as variable name.

4. In **Expression**, configure the expression used to filter the results.

For example, enter the following expression (its return type must be a Boolean):

```
person.age > 30 AND person.city = "New York City"
```

The operator returns every Item of the `Person` type (including the `Employee` type) with an age higher than 30 and living in New York City.

Matching Annotations and Text Scoring

You can select **Advanced > Use matching annotations** to specify matching annotations. These are metadata or explicit.

The query engine computes implicit annotations internally, without explicit declaration in the **Expression**. It

You declare explicit annotations in the **Expression**.

Example

- **Expression:** `person.age > 30 AND ( (person.city == "New York City"){city=1} OR (person.city == "Miami")`
- **Matching annotations:** `city` and `score`

With the explicit matching annotation `city`, the operator annotates the Items that match the query with `city`

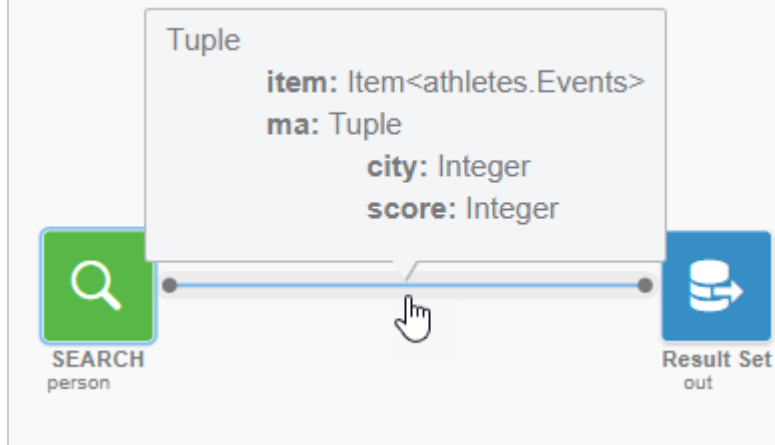
If you add matching annotations, the output type of the operator changes to Tuple. The output displays a T

For the example above, the output is:

Tuple

```
item: Item<athletes.Events>
ma: Tuple
  city: Integer
  score: Integer
```

Tip: You can see this output when hovering over the link coming out of the **SEARCH** operator.



Builtin Matching Annotation

You can use matching annotations to know the frequency of terms in text.

Example 1

Input:

- **Expression:** `Text.hasTerm(item.text, "car")`
- **Matching annotations:** `score`

Output:

Tuple

```
item: Item<core.Item>
ma: Tuple
```

```
score: Integer
```

For example:

```
{
  item: {
    ...
    text: "I love car"
  },
  ma: {
    score: 16
  }
},
{
  item: {
    ...
    text: "I love my car and cars are eveything for me"
  },
  ma: {
    score: 32
  }
},
```

Example 2

Input:

- **Expression:** `Text.hasTerm(item.text, "car"){boost= 10}` OR `Text.hasTerm(item.text, "literature")`
- **Matching annotations:** `score` and `boost`.

Output:

```
Tuple
  item: Item<core.Item>
  ma: Tuple
    score: Integer
    boost: Integer
```

For example:

```
{
  item: {
    ...
    text: "I love car"
  },
  ma: {
    score: 160
  }
}
```

```

    }
  },
  {
    item: {
      ...
      text: "I love car because cars are eveything for me"
    },
    ma: {
      score: 320
    }
  },
  {
    item: {
      ...
      text: "I love litterature"
    },
    ma: {
      score: 42
    }
  },

```

In this example, the scores of the `item.text` containing `car` have been multiplied by `10`, as specified with `{bo`

Other Use of Matching Annotations

You can use matching annotations for other purposes. They are typically useful when used as "side variable

Input:

- **Expression:** `Text.hasTerm(item.text, "red") {matchRed= 1}` OR `Text.hasTerm(item.text, "magenta") {matchMagenta= 1}`
- **Matching annotations:** `matchMagenta` and `matchRed`

Output:

Tuple

```

item: Item<core.Item>,
ma: Tuple
  matchMagenta: Integer
  matchRed: Integer

```

For example:

```

{
  item: {
    ...
    text: "these tulips are red"
  },

```

```

ma: {
  matchMagenta: 0,
  matchRed: 1
},
{
  item: {
    ...
    text: "The ink cartridge color is magenta"
  },
  ma: {
    matchMagenta: 1,
    matchRed: 0
  },
  {
    item: {
      ...
      text: "John cannot make the difference between red and magenta"
    },
    ma: {
      matchMagenta: 1,
      matchRed: 1
    },
  },

```

QUERY CALL

In a data query, you can use the **QUERY CALL** operator to reuse the output of another ☐ data query (or fusion).

This operator is useful to mutualize the configuration of data queries and also share secured queries. It can be used within the same Data Workspace or from a Data Workspace used as dependency.

- [Configuration](#)

Configuration

Configure the operator with the following parameters:

Parameter	Description
Query	Select the ☐ data query or a ☐ data fusion that you want to call. Note: You can call a secured query that does not give direct access to sensitive index data (the Access to Index Unit Data Through Queries).
Result set	If the query is a ☐ data query, you can select the result sets to use in the query call.
Parameters	Optionally, you can see the list of \$ Input parameters defined for the query.

? AGGREGATE

In a data query, you can use the **AGGREGATE** operator to calculate one or several aggregations from the input stream.

For example, you can:

- Count the number of employees.
- Get the sum of a stream of prices.
- Pick the oldest person of a stream of persons.
- Extract the last percentile of a distribution.

You define aggregations with aggregation functions.

You generally need only one aggregation and define it with the **Single aggregator** container.

If you want to process several aggregations at the same time, use the **Multiple aggregator** container.

? DISTINCT

In a data query, you can use the **DISTINCT** operator to remove duplicate elements from the input stream. It relies on a key, which is

This page discusses:

- [Input](#)
- [Output](#)
- [Example](#)

Input

There is one Input. There are no constraints on the element type or on the number of elements in the stream.

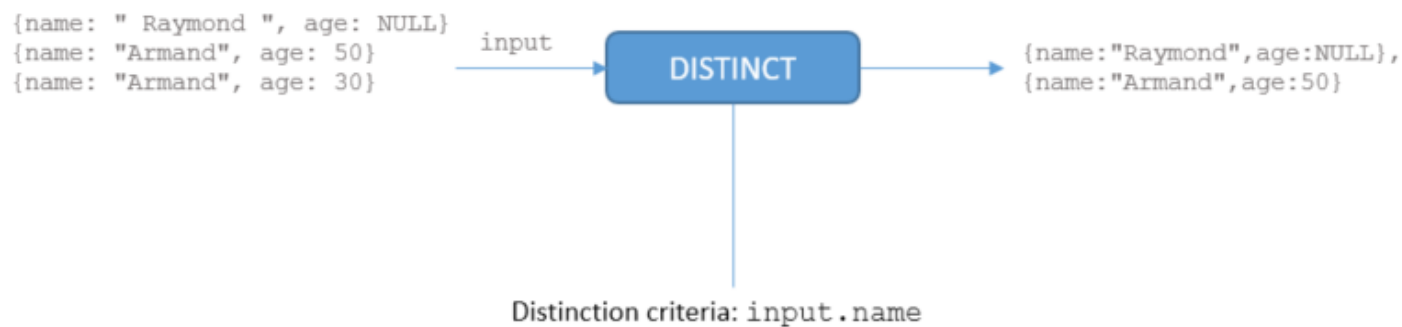
Output

There must be one Output. The element type must be identical to the input.

The output stream must contain as many elements as the input stream. When the input stream is sorted,

Example

Use the name as distinction criteria.



See Also

[UC1 - Get 2 Tables with 2nd Table Showing Details](#)

[Expression Language](#)

[Defining Data Queries](#)

? FILTER

In a data query, you can use the **FILTER** operator to filter elements using a Boolean expression.

If the coming element:

- Matches the expression, it goes through the output.
- Does not match the expression, nothing goes through the output.

This operator is stateless and the context of processing is only 1 element. If the Input Stream is ordered, the order is conserved.

For more information, see the [SEARCH](#) operator and the Expression Language documentation.

This page discusses:

- [Input](#)
- [Output](#)
- [Configuration](#)

Input

There is one Input. There are no constraints on the element type or on the number of elements in the stream.

Output

There must be one Output. The element type is identical to the input.

The output stream contains as many elements as the input stream. It does not affect the order (if any) of

Configuration

Parameter	Type	Comment
Predicate	Boolean expression	Elements for which the Boolean expression is true

See Also

[Expression Language](#)

[Defining Data Queries](#)

GRAPH TRAVERSER

In a data query, you can use the **GRAPH TRAVERSER** operator to process recursive operations on each element of the input stream.

This page discusses:

- [Concepts](#)
- [Example](#)

See Also

[Expression Language](#)

[Defining Data Queries](#)

Concepts

To process your graph of data the way you want, you must define a graph of node processors.

A node processor first receives, and then applies treatments to a node, to finally forward the current node

Node Processors call each other recursively to traverse the whole graph until all nodes have been processed

Traversal Setting

The **GRAPH TRAVERSER** operator can perform two types of graph traversal:

- DFS** (Depth-First Search): The operator always processes the children of a given node before its siblings

- **BFS** (Breadth-First Search): The operator starts at the tree root and explores all sibling nodes at the

Processors

A processor treats a single type of node forwarded by its predecessor. To specify which processor receives

A processor applies the following treatments in this order:

Action	Description
Context updates	Updates context variables (see next sections).
Emit predicate	Expression returning a Boolean. This expression decides whether to call the Emit Expression for
Emit expression	Defines what the GRAPH TRAVERSER operator returns. All emit expressions of all processors m
Stop node condition	Expression returning a Boolean. This expression decides whether to call a Transition for the cu
Transition	Forwards elements to the successor Node processor. For more information, see Transition .

Transition

A Transition is a function that provides elements to forward to the next processor. There are several types

Transition	Description
Explicit Nodes Transition	Expects an expression providing children to the successor Node Processor referenced by To. You can use
Search Transition	Searches items directly from the index (similar to SEARCH operator). Expects an Item class, an Index unit
Conditional Transition	Advanced Transition type that targets 2 processors. It determines the processor to apply at runtime using • <code>true</code>, it uses the main Transition, • <code>false</code>, it uses the Fallback transition

Variables

You can define context variables to store the current state of your traversal. Node processors can access a

There are 3 kinds of variables to answer the different kind of use cases. The differences between these 3 v

Variable type	Description
Global	There is only one instance of this variable for the whole traversal. This instance is visible by everybody. When a No

	Common use case: count the number of processed nodes during the whole traversal.
Path	When a node processor updates a path variable, it copies its new value for each child. This way, the changes made Common use case: For each node, store the path, that is the succession of nodes, from the root that led to this node.
Node	The operator creates an instance of this variable anytime it processes a new node. The 2nd time a node is processed Common use case: Count the number of times a node has been processed during the traversal.

Example

In this example, the graph of data is a social network indexed with the following Data Model:

```
class User {
    name: String;
    friends: List<User>;
}
```

We want to compute all paths that can lead from `UserA` to `UserD`, User A being the root, and UserD the end

```
graph TD
    A[UserA] -->|friends with| B[UserB]
    A[UserA] -->|friends with| C[UserC]
    C[UserC] -->|friends with| D[UserD]
    B[UserB] -->|friends with| D[UserD]
```

The correct output for this use case is `[UserA, UserC, UserD]` and `[UserA, UserB, UserD]`.

To do this, we must configure the **GRAPH TRAVERSER** as follows:

```
graph TD
    A[user] -->|user->friends| A[user]
```

We need:

- A single node processor called `user` treating nodes of type: `Item<User>`
- A **Path** variable to collect graph paths from `UserA` to `UserD` called `path` of type: `List<Item<User>>`.

As the initial processor is `user`, the configuration of the node processor is:

- **Context update:** `path = [...path, user]`
- **Emit predicate:** `user == UserD`
- **Emit expression:** `path`
- **Explicit node transition:** `user->friends` targeting **To:** `user`

In this example, relationships are oriented and `UserD` has no children, so the traversal stops, because there

In a more realistic model, relationships are nonoriented and you can find cycles in the graph. For example

- A **Node** variable: `visited: Boolean = false` that flags whether a node has been processed or not.
- If you add the **Stop node condition** to `user`, the traversal stops on each path if the current node has

In a data query, you can use the **GROUP** operator to encapsulate a subprocessing graph. It is useful for graph factorization to avoid c

This page discusses:

- [Input](#)
- [Output](#)
- [Configuration](#)

Input

This operator can take as input one or several streams. For example, the output of a **SEARCH** operator.

Output

The **GROUP** operator returns one or several streams, depending on the number of graph outputs coming out of the subprocess. The output can be any supported type.

Configuration

Parameter	Comment
Subprocessing graph configuration	Click Edit to open the subprocessing graph configuration and define a subgraph. It can contain one or more streams of operators of various types, with: • An Input operator, • One or more transformation operators, • A Graph Output operator.
Parameters binding	Specify the values of the Input parameters that you want to inject in the subgraph.

See Also

[Types](#)

[Creating Input Parameters](#)

[Expression Language](#)

[Defining Data Queries](#)

 **GROUP BY**

In a data query, you can use the **GROUP BY** operator to group elements from the input stream and compute aggregations on these groups.

This page discusses:

- [Partitioners](#)
- [Aggregators](#)
- [Output](#)
- [Example](#)

Partitioners

You specify how to partition the input stream with partitioning function objects. Define how you want to split the input stream.

You can partition your data with multiple partitioners. When you do so, it applies the partitioners at the same time.

Example:

```
[
  {age:21,surname:"martin",firstName:"Jean"},
  {age:20,surname:"martin",firstName:"Pierre"},
  {age:21,surname:"jack",firstName:"Paul"},
  {age:20,surname:"jack",firstName:"Leo"},
  {age:20,surname:"jack",firstName:"Jeanne"},
]
```

To split with two value partitioners `surname` and `age`, you must have 4 data packages:

- First package:

```
{age:21,surname:"martin",firstName:"Jean"}
```

- Second package:

```
{age:20,surname:"martin",firstName:"Pierre"},
```

- Third package:

- ```
{age:20,surname:"jack",firstName:"Leo"},
```

```
{age:20,surname:"jack",firstName:"Jeanne"},
```

- Fourth package:

```
{age:21,surname:"jack",firstName:"Paul"},
```

The name of the partitioner is the variable name you can use in the `outputExpression`.

## Aggregators

As for the `AGGREGATE` operator, you define the aggregations with **Aggregator**.

## Output

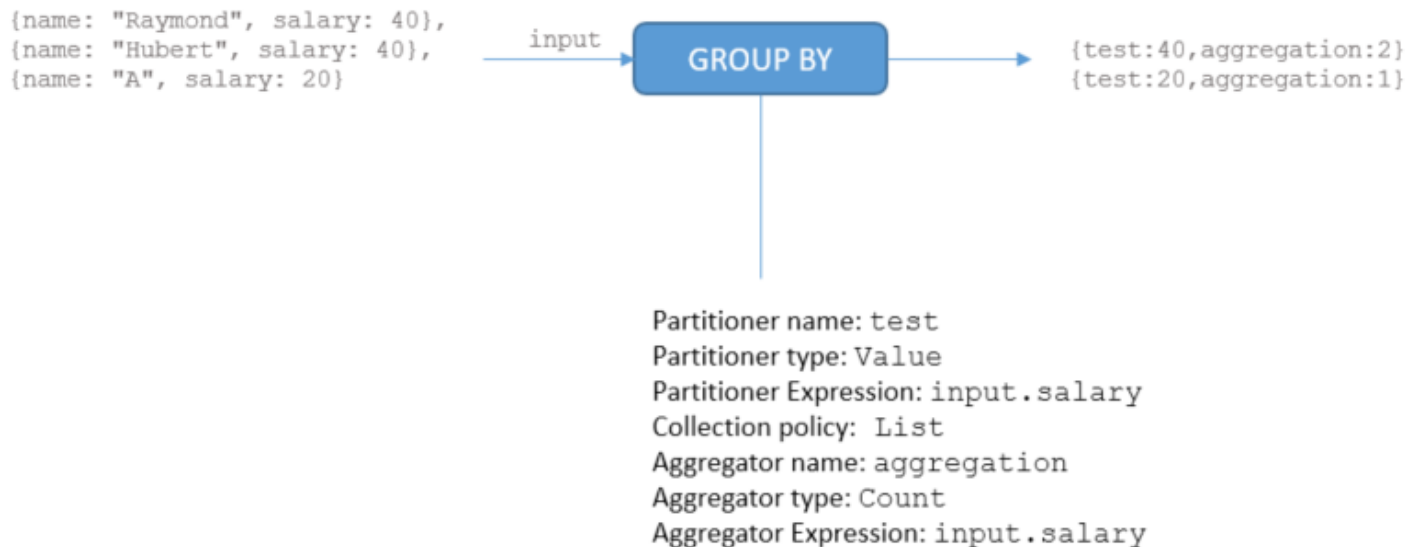
The output is a stream of type: `Tuple<KEY, AGGREG1, AGGREG2, ...>`

The output stream cardinality corresponds to the number of groups found during the grouping operation.

If you define both fast path aggregators and nested graphs, the output type is: `Tuple<KEY, AGGREGFASTPATH1, AGGREGFASTPATH2, ...>`

## Example

You want to group employees who share the same salary.



See Also

[Aggregation Functions](#)

[Partitioners](#)

[UC2 - Get Sum of Values for an Entry](#)

[Expression Language](#)

[?](#) JOIN

In a data query, you can use the **JOIN** operator to join two input streams together in a single output stream. There are different types. If the left input stream is ordered, the output keeps this order. If the right input stream is ordered, this order is kept within a same left element.

**Important:** To join more than 2 data sources you can only use the **Inner join** type.

This page discusses:

- [Configuration Parameters](#)
- [Common Example](#)
- [Left Join](#)
- [Inner Join](#)
- [Full Outer Join](#)
- [Left Excluding Join](#)

See Also

[Expression Language](#)

[Defining Data Queries](#)

## Configuration Parameters

| Parameter         | Comment                                                                                                                                                |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Join clause       | Specifies the join criteria.                                                                                                                           |
| Type              | Specifies the join strategy: <b>Left</b> , <b>Inner</b> , <b>Full Outer</b> , <b>Left Excluding</b> .<br>Default: <b>Left</b> .                        |
| Aggregate         | If <code>true</code> , Data Perspective Studio groups the right elements inside a list for one matching left element.<br>Default: <code>false</code> . |
| Output expression | Optional. Use the output expression to format the result. It operates as a <b>MAP</b> operator placed at the end of the query.                         |

## Common Example

The **Join clause** expression is: `left.company_id == right.company_id`

**Table 1. Left Input**

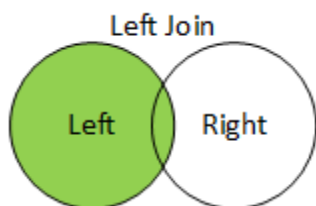
| company_id | country   |
|------------|-----------|
| 1          | Great Bri |
| 2          | France    |
| 3          | Germany   |

**Table 2. Right Input**

| company_id | name   |
|------------|--------|
| 1          | John   |
| 1          | Lucy   |
| 2          | Michel |
| 4          | Mary   |

## Left Join

The **Left** join iterates on left input elements and associates them with right input elements for which the c



Left Join Result if Aggregate=true

See [Common Example](#).

If **Aggregate** is `true`, the **Left** join gives the following result:

| company_id | country       |
|------------|---------------|
| 1          | Great Britain |

|   |         |
|---|---------|
| 2 | France  |
| 3 | Germany |

Data Perspective Studio generates the following result in JSON:

```
{left: {company_id: "1", country: "Great Britain"}, right: [{company_id: "1", name: "John", age: "35"},
{left: {company_id: "2", country: "France"}, right: [company_id: "2", name: "Michel", age: NULL]}
{left: {company_id: "3", country: "Germany"}, right: [company_id: "3", name: NULL, age: NULL]}
```

Left Join Result if Aggregate=false

See [Common Example](#).

If **Aggregate** is `false`, the **Left** join gives the following result:

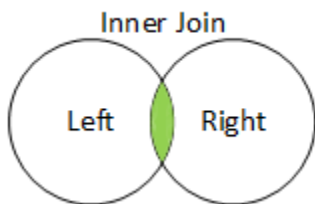
| company_id | country       |
|------------|---------------|
| 1          | Great Britain |
| 1          | Great Britain |
| 2          | France        |
| 3          | Germany       |

Data Perspective Studio generates the following result in JSON:

```
{left: {company_id: "1", country: "Great Britain"}, right: {company_id: "1", name: "John", age: "35"}}
{left: {company_id: "1", country: "Great Britain"}, right: {company_id: "1", name: "Lucy", age: "42"}}
{left: {company_id: "2", country: "France"}, right: {company_id: "2", name: "Michel", age: NULL}}
{left: {company_id: "3", country: "Germany"}, right: {company_id: "3", name: NULL, age: NULL}}
```

## Inner Join

The **Inner join** takes the intersection of the left and right input.



**Important:** To join more than 2 data sources you can only use the **Inner join** type.

### Inner Join Result

See [Common Example](#).

The **Inner** join gives the following result:

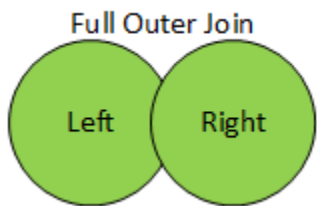
| company_id | country       |
|------------|---------------|
| 1          | Great Britain |
| 1          | Great Britain |
| 2          | France        |

Data Perspective Studio generates the following result in JSON:

```
{left: {company_id: "1", country: "Great Britain"}, right: {company_id: "1", name: "John", age: "35"}}
{left: {company_id: "1", country: "Great Britain"}, right: {company_id: "1", name: "Lucy", age: "42"}}
{left: {company_id: "2", country: "France"}, right: {company_id: "2", name: "Michel", age: NULL}}
```

### Full Outer Join

The **Full Outer** join returns all elements.



### Full Outer Join Result

See [Common Example](#).

The **Full Outer** join gives the following result:

| company_id | country       |
|------------|---------------|
| 1          | Great Britain |
| 1          | Great Britain |
| 2          | France        |
| 3          | Germany       |

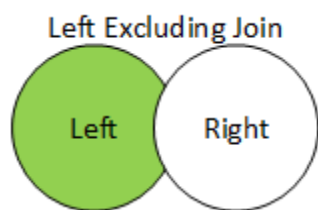
|   |      |
|---|------|
| 4 | NULL |
|---|------|

Data Perspective Studio generates the following result in JSON:

```
{left: {company_id: "1", country: "Great Britain"}, right: {company_id: "1", name: "John", age: "35"}}
{left: {company_id: "1", country: "Great Britain"}, right: {company_id: "1", name: "Lucy", age: "42"}}
{left: {company_id: "2", country: "France"}, right: {company_id: "2", name: "Michel", age: NULL}}
{left: {company_id: "3", country: "Germany"}, right: {company_id: "3", name: NULL, age: NULL}}
{left: {company_id: "4", country: NULL}, right: {company_id: "4", name: "Mary", age: "24"}}
```

## Left Excluding Join

The **Left Excluding** join retrieves the left input elements that do not match any right input elements.



Left Excluding Join Result

See [Common Example](#).

The **Left Excluding** join gives the following result:

| company_id | country |
|------------|---------|
| 3          | Germany |

Data Perspective Studio generates the following result in JSON:

```
{left: {company_id: "3", country: "Germany"}, right: {company_id: "3", name: NULL, age: NULL}}
```

## ? LIMIT

In a data query, you can use the **LIMIT** operator to limit or skip a number of elements in the stream.

This page discusses:

- [Input](#)
- [Output](#)
- [Example](#)

## Input

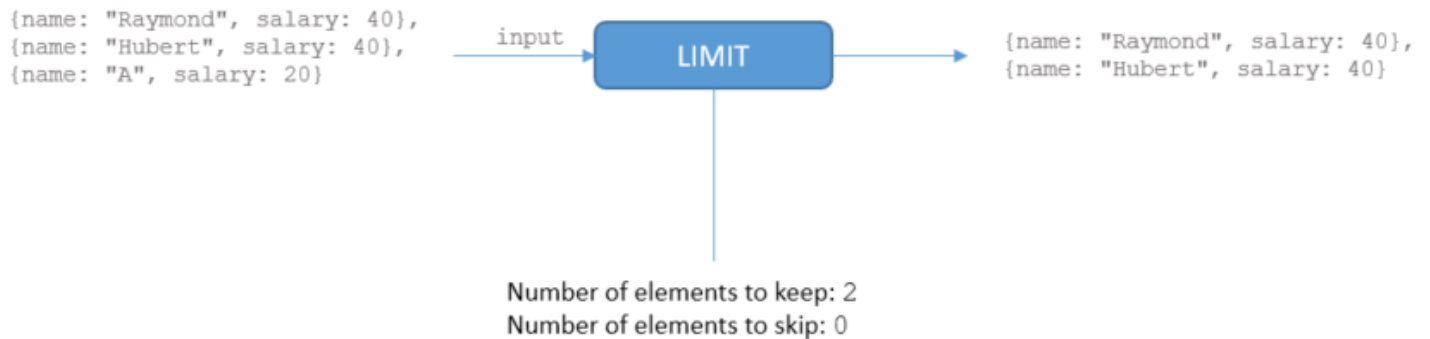
There is one Input. There are no constraints on the element type or on the number of elements in the stream.

## Output

There is one Output. The elements and their order are modified, only the number of elements changes.

## Example

In the following example, the operator limits the number of elements to 2.



See Also

[Expression Language](#)

[Defining Data Queries](#)

## ? MAP

In a data query, you can use the **MAP** operator to transform the elements of the stream. It is a 1 to 1 operator, which means that it receives one element and produces one element. By default the operator is stateless, so the processing context is made of one element only. Use the **Use state** option for cases that require state.

This page discusses:

- [Configuration](#)
- [Stateful Mode](#)
- [Subprocessing Graph](#)
- [Use the Operator to Clean Elements](#)
- [Examples](#)

See Also

# Configuration

The configuration of this operator is one single expression with no constraints on the output type. The output is a stream of elements.

## Stateful Mode

The stateful MAP iterates over each element sequentially, and keeps previous results to process the next element.

Use this operator to compute things like moving average, derivation, etc.

The pseudo code for this operator is the following:

```
var element;

var var1 = ...// default value

var var2 = ...// default value

var var3 = ...// default value

while((element = getNextValue()) != null && $termination_expression == false) {

 var1 = ... //update var

 var2 = ... //update var

 var3 = ... //update var

 emit($output_expression);

}
```

**Note:** As for now, the operator emits an element for every element of the input stream. If you need fewer elements, you can use the `$limit` parameter.

| Parameter | Type                         | Comment                                                                                                                                                                                                                                                                               |
|-----------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Variables | List of variable declaration | List of variables available during the iteration and then in the Output.<br>Parameters: <ul style="list-style-type: none"><li><b>Name:</b> enter the variable name.</li><li><b>Type:</b> select <b>String</b>.</li><li><b>Default value:</b> Default value of the variable.</li></ul> |
| Updates   | List of variable updates     | List of assignments used to update the variables.                                                                                                                                                                                                                                     |

|        |                   |                                                                                                                                                                                                                                                                                                                                                                     |
|--------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        |                   | <p><b>Note:</b> You can assign a variable more than one time in the same iteration.</p> <p>The order matters as the last version of the variable is available in the iteration.</p> <p>Parameters:</p> <ul style="list-style-type: none"><li>• <b>Name:</b> enter the variable name.</li><li>• <b>Expression:</b> Expression used to update the variable.</li></ul> |
| Output | Output expression | Expression used to build the element that the operator emits. The variable <b>input</b> is available in the expression.                                                                                                                                                                                                                                             |

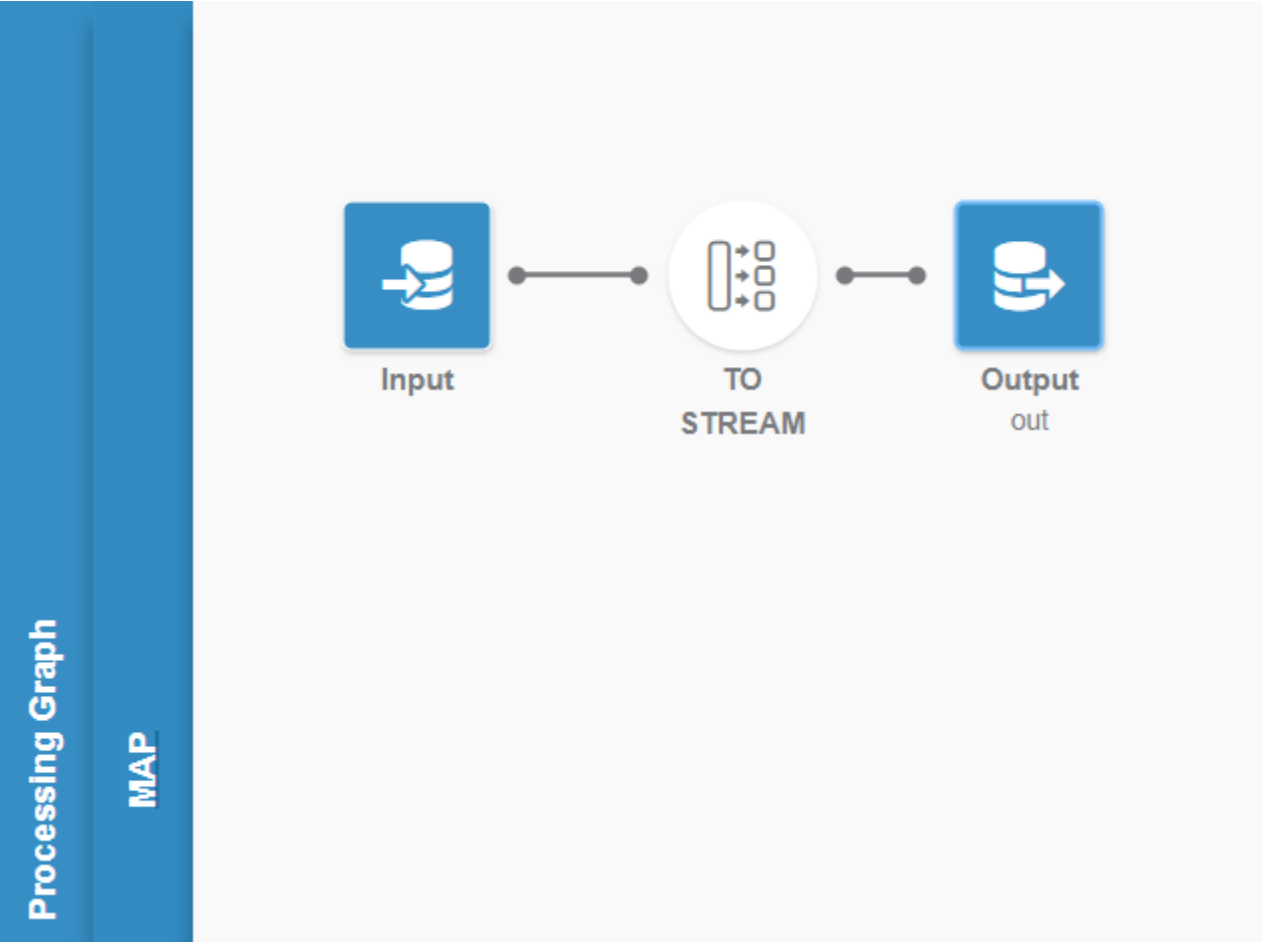
## Subprocessing Graph

Select **Use sub processing graph** to define a subgraph to apply on each input element. This is useful to control the processing of each element.

For example, if you have a dataset of tickets, where each ticket contains a list of comments, you might want to process each comment in a subgraph, with:

1. An **Input** operator, with a **Link to** property referencing the **MAP** operator **Operator input variable**.
2. One or more transformation operators to process the comments, like a **TO STREAM** operator to retransform the comments into a stream.
3. A **Graph Output** operator, that you can use as a variable in the **Output Expression** of the parent **MAP** operator.

**Note:** In the **MAP** operator **Output Expression**, you typically define a tuple with an input element and a stream of transformed elements.



Graph Input

By default, each subgraph has an **Input** operator, which receives all the input elements on which to apply

| Parameter | Description                                                                                                      |
|-----------|------------------------------------------------------------------------------------------------------------------|
| Link to   | References the input variable.                                                                                   |
| Type      | Allows you to specify the type to extend in the input (for example, <code>Item&lt;persons.Employee&gt;</code> ). |

Graph Output

For most cases, you must end the subgraph with a **Graph Output** operator. This operator contains the vari

Use the Operator to Clean Elements

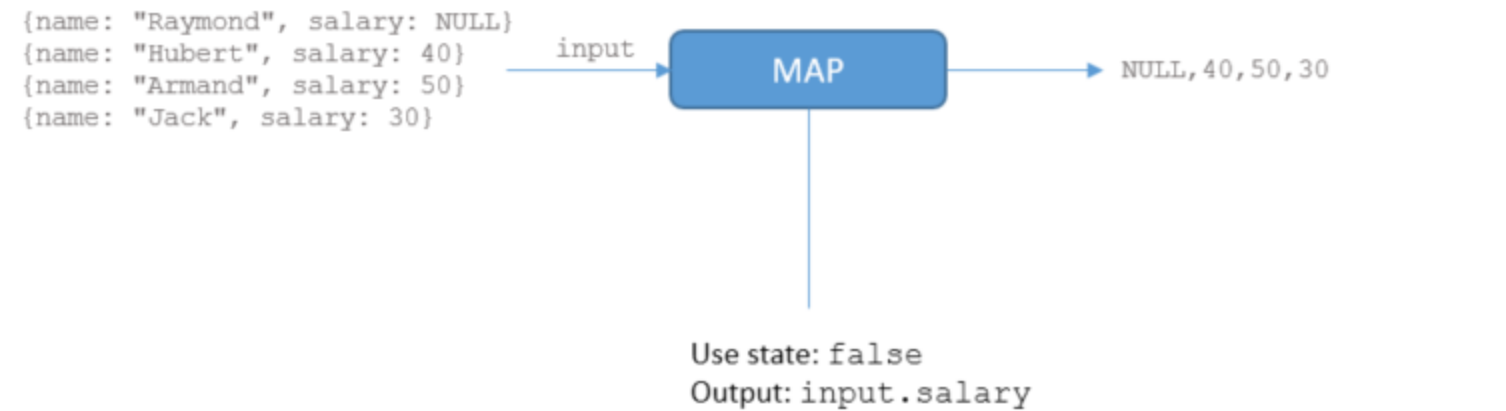
You can use the **MAP** operator to "clean" elements by removing attributes that are not used further in the  
An Item in a stream is just a reference inside the Index data structure. When you put an Item attribute in

Examples

A basic case is to compute something using existing values of the element.  
For example, if the input elements are of type `Tuple<income:Float, taxes_rate:Float>`, you can output a new  
`{income:element.income, taxes_rate:element.taxes_rate, net_income:income*(1-taxes_rate)}`  
**Note:** Using the `spread` operator, you can write this in a shorter way: `{...element, net_income:income*(1-ta`

Using more advanced features from the expression language, you can configure the **MAP** operator to do p  
In the following example, we use the **MAP** operator to make a JOIN (LEFT) through a reference between a

`{manager:manager, employees:List.sort(manager<-[corporate.Employee.manager], (employee)->employee.birth`  
In this example, we want to create a map for salaries.



In a data query, you can use the **RANK** operator to associate rank levels to each element of the stream. It does not sort the stream of elements. For example, if your stream contains: a, b, c, d, e, and your **RANK** operator has a sorter defined by descending order, you still have a, b, c, d, e. As for the **SORT** operator, you can specify multiple sorters to rank elements according to multiple attributes.

**Note:** The configuration parameters are the same as for the [SORT](#) operator.

This page discusses:

- [Input](#)
- [Output](#)
- [Example](#)

## Input

There is one Input. There are no constraints on the element type or on the number of elements in the stream.

## Output

There is one Output. The output stream counts as many elements as the input stream counts, and the order is the same as the input.

The type of the elements is a Tuple containing the rank (Integer) and the original element.

## Example

In this example, we associate a rank to every person regarding their age.

The result is a stream of Tuples, each containing a person and a rank.



See Also

[Expression Language](#)

# ? SORT

In a data query, you can use the **SORT** operator to order elements in a meaningful way.

By default, the output of a data query is displayed in no particular order. To order elements in a meaningful way, you have two options in the processing graph of your data query.

This page discusses:

- [RESULT SET Order VS SORT Operator](#)
- [Sorter Configuration](#)
- [Input](#)
- [Output](#)
- [Example](#)

## RESULT SET Order VS SORT Operator

If defined, the [RESULT SET](#) order has always precedence over **SORT** operators. That is why you should always use the **SORT** operator.

## Sorter Configuration

Whenever you set an order in the [RESULT SET](#) or use the **SORT** operator, the configuration is the same: `expression=, orderAsc=, nullsLast=`

| Sorter Type     | Description                                                                                                                                                                                                                                                                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Value Sorter    | <p>Works as in SQL and asks for 3 elements:</p> <ul style="list-style-type: none"><li>• A sort expression to get the comparable value.</li><li>• The sort order: ASC/DESC.</li><li>• Handle NULL elements: LAST/FIRST.</li></ul> <p>For example, to sort a list of tickets by rising id:</p> <pre>expression=ticket.id, orderAsc=true, nullsLast=true</pre> |
| Explicit Sorter | <p>Takes an expression that extracts a value, and another one to specify this value order.</p> <p>For example, to sort tickets by status ("new" &lt; "qa" &lt; "closed"):</p> <pre>expression=ticket.status, order=["new", "qa", "closed"],<br/>orderAsc=true, nullsLast=true</pre>                                                                         |

To sort by multiple attributes, specify a list of sorters. You can sort by an attribute in ascending order and then by another attribute in descending order.

For example, to sort tickets by status and then by descending ticket id:

- Add an explicit sorter:
- ```
expression=ticket.status, order=["new", "qa", "closed"],
```

```
orderAsc=true, nullsLast=true
```

- Then add a value sorter:

```
expression=ticket.id, orderAsc=false, nullsLast=true
```

Input

There is one Input. There are no constraints on the element type or on the number of elements in the stream.

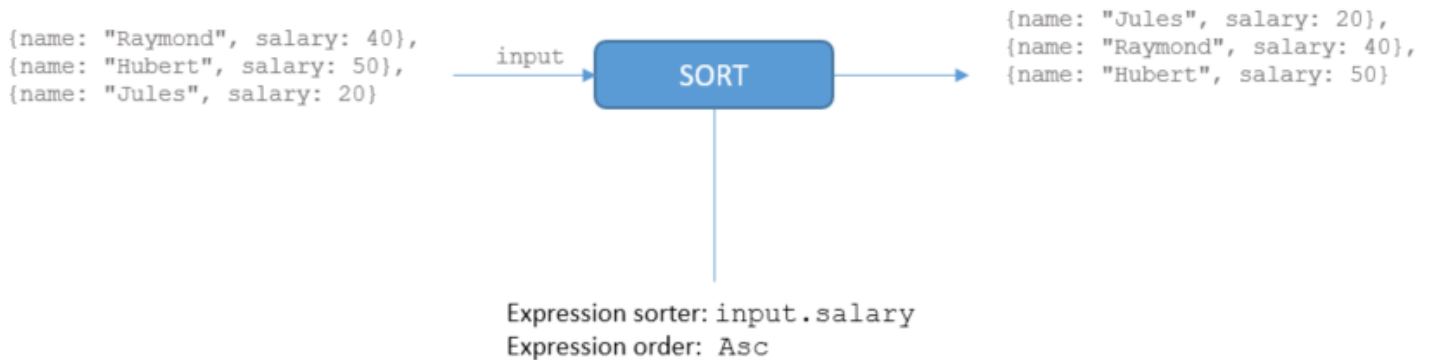
Output

There is one Output. The output stream counts as many elements as the input stream. The elements are ordered.

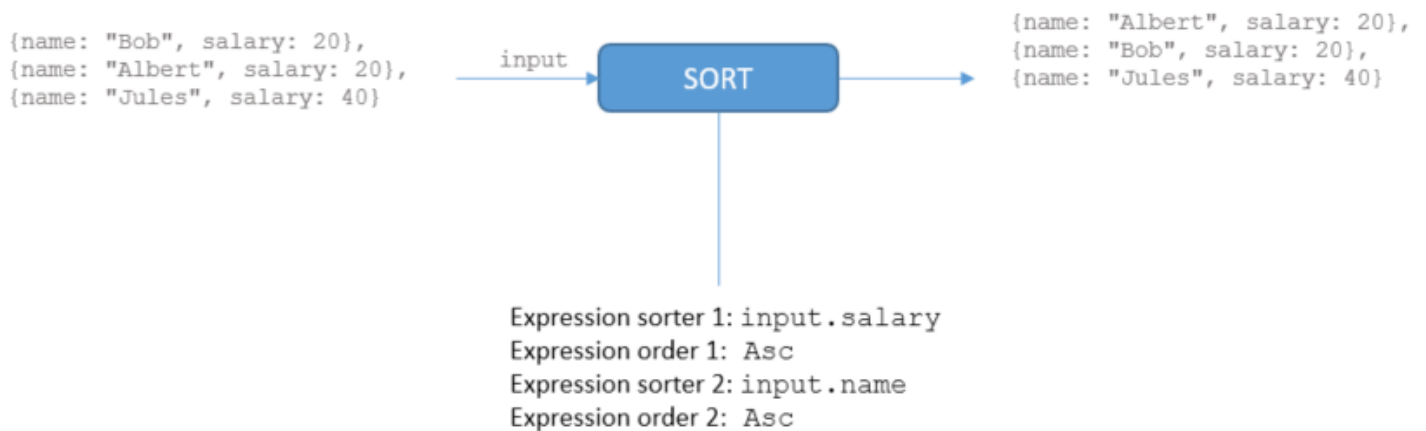
Example

In this example, we sort the employee by salary.

The output stream is ordered.



Multiple Sorters



Important: With a **SORT** operator, the result order can change when the DataQuery is run again if several persons have the same salary. Choose the "Stable" option when the Data Query is run again. Choose this option if pagination is used on a ResultSet, to avoid having elements ordered in a different way.

See Also

[Expression Language](#)

[Defining Data Queries](#)

? SWITCH

In a data query, you can use the **SWITCH** operator to apply different treatments to different groups.

The **SWITCH** operator takes a single stream and returns several streams, each containing elements sharing the same expression evaluation.

This page discusses:

- [Input](#)
- [Output](#)
- [Example](#)

Input

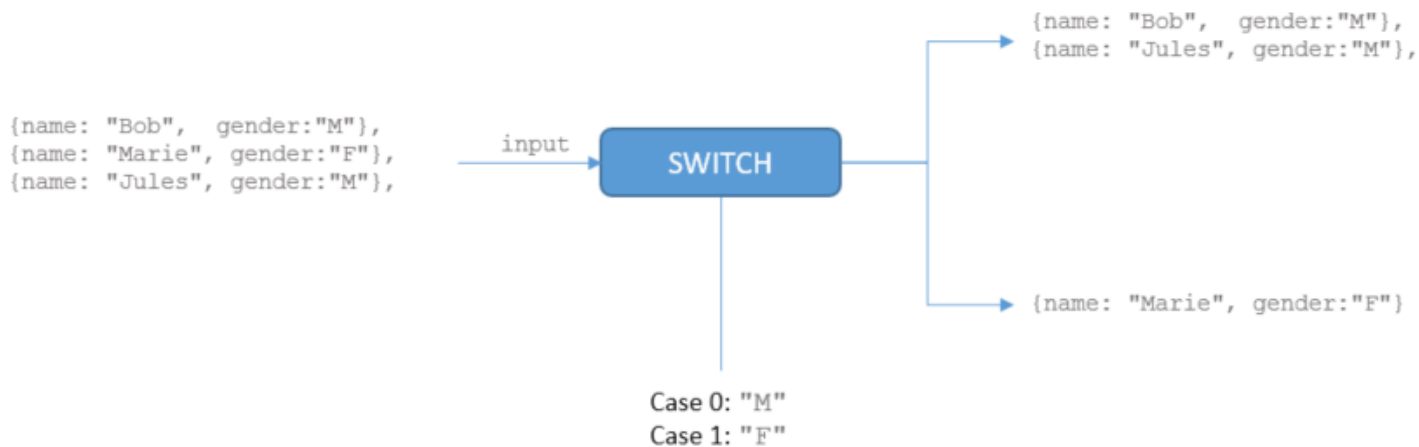
A stream of elements.

Output

N streams of E elements. Each stream contains elements sharing the same expression evaluation. The number of streams is N.

Example

From one stream with a list of employees, you want to return 2 streams, based on their genders.



See Also

[Expression Language](#)

[Defining Data Queries](#)

[?](#) TO STREAM

In a data query, you can use the **TO STREAM** operator to generate a stream of elements from a list.

This page discusses:

- [Input](#)
- [Output](#)
- [Configuration](#)

Input

In a stream of E elements, each element contains a list of F elements.

Output

For each E element, the output returns a list of F elements. If the input element is NULL, the output for this

Configuration

Parameter	Description
Operator input variable	Specify the name to give to the variable that contains the input data. The variable is then available when you c
Collection to flatten	Specify the list attribute to flatten. For each input element, the operator retrieves the list and outputs each ele list). Example: You are querying friends and you want to get a stream with their names. Collection to flatten: <code>item.friends</code> Input elements: • <code>{name: "Bob", friends: ["Lea", "Alice", "Betty"]}</code> • <code>{name: "Albert", friends: ["John"]}</code> Output elements: • <code>"Lea"</code> • <code>"Alice"</code> • <code>"Betty"</code>

	• "John"
Output expression	Optional. Configure an expression to transform the elements before the output. The goal is to combine the input element specified in the Operator input variable with the output element represented by the Output expression to produce the transformed output. Example: You have a list of persons linked with their friends, and you want to generate pairs of friends in the output. Collection to flatten: <code>item.friends</code> Output expression: <code>{person: item.name, friend: value}</code> Input elements: • <code>{name: "Bob", friends: ["Lea", "Alice", "Betty"]}</code> • <code>{name: "Albert", friends: ["John"]}</code> Output elements: • <code>{person:"Bob", friend:"Lea"}</code> • <code>{person:"Bob", friend:"Alice"}</code> • <code>{person:"Bob", friend:"Betty"}</code> • <code>{person:"Albert", friend:"John"}</code>

See Also

[Expression Language](#)

[Defining Data Queries](#)

[UNION](#)

In a data query, you can use the **UNION** operator to concatenate the values of two different input streams in a single output stream.

This page discusses:

- [Input](#)
- [Output](#)
- [Example](#)

Input

N subtypes of E elements.

Output

A subtype of E elements, which are the common subtype of all inputs.

Important: Unlike the mathematical Union, duplicates are not removed. The union operator does not keep the order of the input elements.

Example

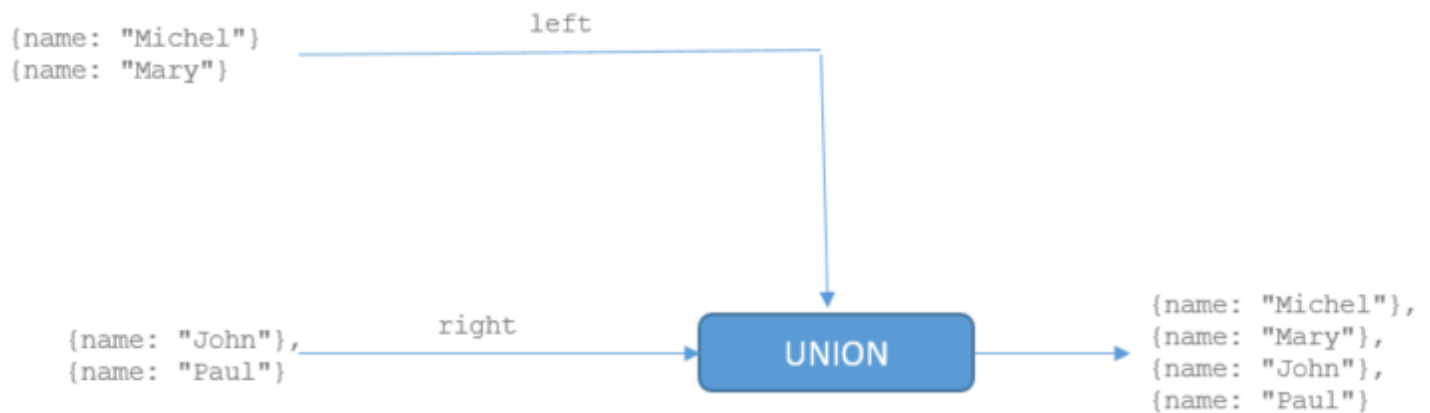
If you have 2 classes:

- `class B {}`
- `class A : B {}`

... as `Item<B>` is a subclass of `Item<A>`, you can make a union that returns `Item<B>`.

For example:

- The union between `Person` and `Company` does not work, because `Person` is not a subtype of `Company`, and
- The union between `core.Item` and `Person` works fine, as `core.Item` is a subtype of `Person`.



See Also

[Expression Language](#)

[Defining Data Queries](#)

[?](#) TO VARIABLE

In a data query, you can use the **TO VARIABLE** operator to collect the stream and store it into a graph variable. You can then call this variable in another part of the query. The execution of the service is only possible if there is no dependency cycle between these variables.

Warning: This mechanism can lead to accumulation or to the re-execution of a graph part. This can increase memory usage and impact the configuration.

- This page discusses:
- [Configuration](#)
 - [Input](#)
 - [Output](#)
 - [Example](#)

Configuration

Parameter	Type	Description
Variable name	String	Name of the graph variable to create. Important: The name must not include @

Input

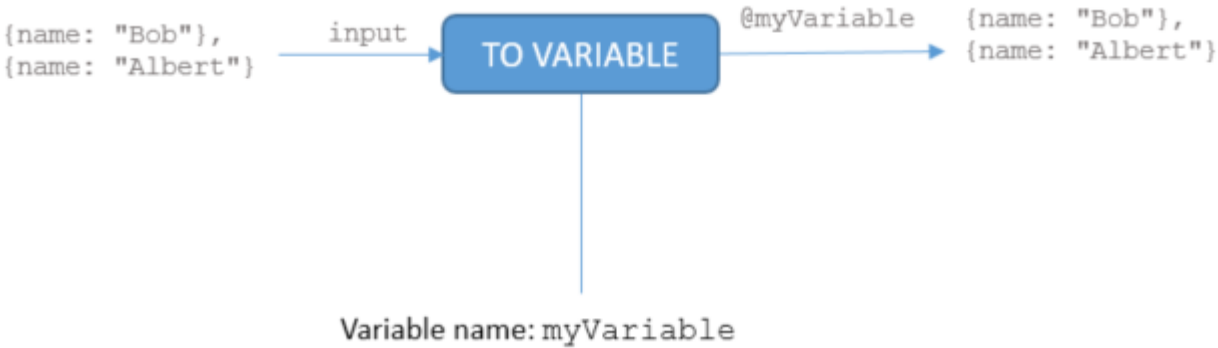
There is one Input. There are no constraints on the element type. The stream can contain more than one element.

Output

No output stream. This operator creates a graph variable.

Example

You can store information into a variable to reuse it in another operator.



See Also

[ER SECURITY](#)

[Expression Language](#)

[Defining Data Queries](#)

RESULT SET

You can use the **RESULT SET** operator at the end of a data query to configure its output. It has many features to help you format your output. A **RESULT SET** operator must have one or several **Views**.

This page discusses:

- [General Parameters](#)
- [Primary Fields](#)
- [Dimensions](#)
- [Aggregations](#)
- [Filters](#)
- [Views](#)
- [Examples](#)

General Parameters

Parameter	Description
Name	Specify the RESULT SET name. This is the name that you call in your visualizations.
Input name	Specify the name to give to the variable that contains the input data. The variable is then available when you use expressions.
Enabled	Activates or deactivates the RESULT SET .
Predicate parameter	This data query parameter stores the filters applied to this RESULT SET . It allows you to use the same filters in multiple RESULT SET s.

Primary Fields

Fields are the most basic elements in a **RESULT SET**. They contain a name and an expression. The expression extracts a value from each element of the input stream. You can specify the fields to use as **Primary fields** in the [Dimensions](#), [Filters](#), or [Table View](#). For example, if the **RESULT SET** receives a stream of persons, you can define a **Field** called `fullname` with the expression `person.fullname`.

Dimensions

A **Dimension** is the combination of a class, a **Field**, and a partitioning function (which allows you to partition data).
Use the **Collection policy** parameter if the field has a collection type (`Collection`, `List`, or `Map`), to define if the field is a dimension.
For example, you can have a dimension referencing an `age` field and a partitioning function that partitions data by age.

Important: Dimensions are used by [Cube View](#) and [Filters](#).

Aggregations

An **Aggregation** is the equivalent of a measure. For example, the height of a bar in a bar chart, or the color of a bar.
Technically, an aggregation is a function that provides a value for each partition defined by dimensions.

Filters

Use filters to filter out a dataset. For example, if you only want to show data for groups of persons who are male.

Important: If you create filters: - At the RESULT SET level, they impact all views. - At a specific view level, they impact this specific view only.	
Filter	Description
Expression Filter	To filter on a Field with an expression. Select Update ResultSet predicate to update the Predicate parameter defined in the General Parameters .
Values Field Filter	To filter on a Field by defining one or several Values . Select Update ResultSet predicate to update the Predicate parameter defined in the General Parameters .
Range Field Filter	To filter on a Field by defining Ranges . For example, if you have grouped persons by age ranges of 10 years with a partitioning function (0-10, 10-20, etc.), you get as output 10-20 and 20-30, but nobody in these two groups is less than 12 or more than 25 years-old. Select Update ResultSet predicate to update the Predicate parameter defined in the General Parameters .
Values Dimension Filter	To filter on a Dimension (on groups) by defining one or several values. For example, if you have grouped persons by age ranges of 10 years with a partitioning function (0-10, 10-20, etc.), you get as output 10-20 and 20-30, but nobody in these two groups is less than 12 or more than 25 years-old. Select Update ResultSet predicate to update the Predicate parameter defined in the General Parameters .
Tree Dimension Filter	To filter on a Dimension (on groups) by defining one or several TreePathFilters . In each Path Filter , you can define a Path prefix and a Max descendant depth (a length). An input path matches a filter Path , if the filter Path is a prefix of the input path, and the additional part of the input path has a length less than or equal to the Max descendant depth . For example, if you have a file in your file system with the input path <code>C:\users\Doc\Folder\File</code> and the following filter: Path=<code>C:\users\Doc</code> and Max descendant depth=<code>1</code>, the filter does apply, and the input does not match the filter.

	• Path=C:\users and Max descendant depth=4, the filter does apply, and the input matches the filter condition. • Path=D:\users and Max descendant depth=1, the filter is not a prefix, so it does not apply. This Tree Dimension Filter can filter out the input path.
--	--

Views

A view represents a projection of your data. Usually you may need one view per graph or visualization displayed.

There are three kinds of views, and each view is suitable for different use cases. The different views share the same settings as filtering criteria in the [Raw view](#).

For each view, you can define **Sorters** to order elements in a meaningful way. For more information, see [Sorters](#).

View	Description
Raw view	This view returns data without any formatting and requires a few settings only. Use this view to show: • Output data without any complex settings. • Polymorphic data, which is a stream of elements that do not support the same attributes. Tip: The Raw view is the simplest one. You can start with that view if you want to use the RESULTSET operator quickly.
Table view	This view formats data in a table. Each field gives a column in output (see Primary Fields). The Table view provides filtering and sort features that are based on the columns. Use this view to show: • Data in a table visualization. • Data with a known structure.
Cube view	This view creates partitions of your data and processes aggregations on them. A Cube view is made of Axes and Aggregations. An axis is a very visual concept. For example, a heatmap, or a risk matrix, contain two axes, because data are represented in a 2D space. A line chart has only one axis. For most cases, you may use one axis, sometimes two axes, and almost never more than two axes. An Axis provides pagination and sort features. Each axis must have at least one Dimension. You can define several dimensions on one axis to create a hierarchy, or group elements. • One axis with one dimension of cities would return: Berlin > New York > Lyon > Munich > Paris • One axis with two dimensions (one for countries and one for cities) would return: USA/New York > France/Lyon > Germany/Berlin Use this view to show data in charts. For example, line chart, pie chart, heatmap, etc.

Examples

In the following examples, we have a **RESULT SET** with an input stream of tasks.

Example	Description
Table of tasks	To show the list of tasks in a table with 3 columns (name, priority, project) • <code>name: tasks.name</code> • <code>priority: task.priority</code> • <code>project: task.project</code>
Bar chart with number of tasks per project	To have a bar chart showing the number of tasks per project • An axis with one dimension using a <code>Value</code> partitioner • An aggregation that counts the number of tasks
Heat map of number of tasks per project and per priority	To have a heat map showing the number of tasks per project and per priority • A first axis with one dimension using a <code>Value</code> partitioner • A second axis with one dimension using a <code>Value</code> partitioner • An aggregation that counts the number of tasks

See Also

[Aggregation Functions](#)

[Partitioners](#)

[SORT](#)

[Expression Language](#)

[Defining Data Queries](#)

[?](#) Expression Language Principles

This topic presents the main principles of the Expression Language syntax.

This page discusses:

- [Learn by Example](#)
- [Call a Function Specific to a Namespace](#)
- [Create a Tuple \(Composite Object\)](#)
- [Append an Element to a List](#)
- [Navigate through Item References](#)
- [Find an Element in a List](#)
- [Sanitize NULL Values](#)
- [Make a String Concatenation](#)

See Also

[Types](#)

[Variables](#)

[Input Parameters](#)

[Expression Operators](#)

[Functions](#)

[Lambda Functions](#)

Learn by Example

Let us start with a Boolean expression:

```
person.firstName == "Bob" AND (NOT person.age < 18)
```

In this expression, `person` is a variable.

The use of logical expression operators (`AND`, `NOT`) is similar to the use of logical operators in SQL.

Tip: To add comments in your expressions, use `/* */` or `//`.

Call a Function Specific to a Namespace

To call a function specific to a namespace, use the syntax `<Namespace>.<functionName>`:

```
/* Here `Text` is the namespace and `hasTerm` the specific function that will look for occurrences of "java" in the resume */
Text.hasTerm(person.resume, "java")
```

For more information about full-text search functions, see [Functions](#).

Create a Tuple (Composite Object)

To create a tuple, use the following syntax:

```
{name: person.firstName, bmi: person.weight / Math.pow(person.height, 2)}
```

Important: Tuple names must not contain any blank spaces.

The tuple looks like a JSON-object notation, and behaves like 'structs' in C. For more information, see [Tuple](#).

Append an Element to a List

To append an element to a list, you can use the spread operator `...`, which performs in-place expansion. The

```
[...person.phones, "0123456789"]
```

For more information, see [List Manipulation](#).

Navigate through Item References

To specify a direct traverse in index unit items, you can use the arrow operator `->`.

For example, to get my grand father's first name, you can use the following expression:

```
person->father->father.firstName
```

For more information, see [Field Access](#).

Find an Element in a List

To find an element in a list, you can use the `List` type with the `anyMatch` function.

```
List.anyMatch(person.phones, phone => phone == "0123456789")
```

You can also call a function directly from an object, using the following function extension:

```
person.phones.anyMatch(phone => phone == "0123456789")
```

Notes:

- See the use of a lambda expression as parameter of the `anyMatch` function.
- Here, the Expression Language infers the type of the lambda parameter. For more information, see [Lambda](#).

Sanitize NULL Values

To test the nullity of an object, you can use the `IS` operator. For more information, see [Comparison](#).

For example, to replace `NULL` values by a default value called `Foo`:

```
person.firstName IS NULL ? "Foo" : (" : person.firstName
```

Here we have a ternary operator that uses the syntax `<CONDITION> ? <EXPRESSION1> : <EXPRESSION2>`:

- If `person.firstName` does not have any value, it returns `Foo`.
- Otherwise, it returns the value of the `person.firstName` variable.

This is equivalent to `person.firstName || "Doh" : ("`

Note: The expression language tries to be as robust as possible at run time to avoid having `NullPointerException`.

- The expression: `1 + NULL` returns `NULL`

- `person.firstName` returns `NULL` if the person is `NULL`

Make a String Concatenation

You can concatenate a string using the `+` binary operator.

For example:

```
person.firstName + " is " + person.age + " years-old"
```

Note: Here, we use blank spaces between double quotes to add spaces between strings. For example, `" is "`

[?](#) Types

The Expression Language is strongly typed.

Note: All types are immutable and nullable.

This page discusses:

- [Numeric Types](#)
- [Date/Time Types](#)
- [Character Types](#)
- [Boolean Type](#)
- [Binary Types](#)
- [Collection Types](#)
- [Composite Types](#)
- [Other Types](#)

Numeric Types

Type	Description
<code>Decimal<precision></code>	Signed 64-byte decimal number. A fixed number of bits that depends on the specified precision are reserved to the right of the decimal point. This type is suitable for monetary calculations as it is not prone to rounding errors, unlike <code>Float</code>. The character
<code>Float</code>	Signed 64-bit floats.
<code>Integer</code>	Signed 64-bit integer.

Date/Time Types

Type	Description
Date	A date (year, month, day). You can use the following properties with this type: • <code>year</code> • <code>month</code> • <code>quarter</code> • <code>weekOfYear</code> • <code>day</code> • <code>dayOfWeek</code> • <code>epochDay</code>
DateTime	A date-time with timezone and millisecond precision (year, month, day, hour, minute, second, millisecond). You can use the following properties with this type: • <code>year</code> • <code>month</code> • <code>quarter</code> • <code>weekOfYear</code> • <code>day</code> • <code>dayOfWeek</code> • <code>epochDay</code> • <code>hour</code> • <code>minute</code> • <code>second</code>
LocalDateTime	A date-time with second precision (year, month, day, hour, minute, second) and without timezone. You can use the following properties with this type: • <code>year</code> • <code>month</code> • <code>quarter</code> • <code>weekOfYear</code> • <code>day</code> • <code>dayOfWeek</code> • <code>epochDay</code> • <code>hour</code> • <code>minute</code> • <code>second</code>
Duration	An amount of time expressed in seconds. You can use the following property with this type: <code>seconds</code>
Instant	A timestamp representing the number of nanoseconds that have elapsed since 00:00:00 UTC on 1 January 2000.

Period	A span of time expressed in years, months, and days. You can use the following properties with this type: - years - months - days
--------	---

Character Types

Type	Description
String	A sequence of characters. You can use the following property with this type: <code>length</code>
TextMatch	Represents the result of text matching functions.

Boolean Type

Type	Description	Literals or constructor
Boolean	Boolean	<code>true</code> / <code>false</code>

Binary Types

Type	Description
Binary	Opaque byte array.
Geometry	A vector geometry object (point, polygon, ...), represented in a well-known binary format (WKB).

Collection Types

Type	Description
Embedding (collection of floats)	An embedding is a float vector (an array of 32-bits floats) that encodes the meaning of words or concepts. Vectors that are "close" to each others are expected to represent similar concepts in meaning. The proximity between two vectors is given by a similarity function. The Semantic Graph Index supports three similarity functions: cosine, dot product, and euclidean.
List<E>	An ordered collection. You can access elements by their index. You can use the following properties with this type: <code>length</code>

	<code>first</code> <code>last</code>
<code>Map<K, V></code>	Key/value dictionary. You can use any type as either key or value. Map keys (that is, <code>K</code>) cannot be <code>NULL</code>. You can use the following properties with this type: <code>length</code> <code>keys</code> <code>values</code> <code>entries</code>
<code>Set<E></code>	Unordered collection with uniqueness guarantee. You can use the following properties with this type: <code>length</code> <code>values</code>
<code>TimeSeries<T></code> (collection of points)	A time series is a series of data points (each data point is the association of a timestamp and a value). You can use the following properties with this type: <code>earliest</code> <code>latest</code> <code>length</code>

Composite Types

Type	Description
<code>Item<pkg.class></code>	Represents an Item coming from an index unit.
Range	Represents two bounds, which can be inclusive or exclusive. A bound can extend to a positive or a negative infinity. A range supports the following types: <code>Integer</code>, <code>Decimal</code>, <code>Float</code>, <code>Date</code>, <code>DateTime</code>, <code>LocalDateTime</code>. You can use the following properties with this type: <code>min</code> <code>max</code>
<code>Tuple<K1: V1, K2: V2, ...></code>	An object that encapsulates values of multiple types. It must contain an ordered list of entries. Define each entry with a name (tuple names must not contain any blank spaces) and a type. For example, the type <code>Tuple<firstname: String, age: Integer></code> defines a tuple that encapsulates <code>firstname</code> and <code>age</code> accessors.

Other Types

Type	Description
Any	Any type.
Comparable	Any type that you can use to sort on (<code>Integer</code> , <code>Float</code> , <code>String</code> , ...). You typically use this type in a <code>SORT</code> or a <code>RANK</code> operator.
<code>Function<ParameterType1, ..., ReturnType></code>	Lambda function.
Null	Type used for <code>NULL</code> value when needed.

See Also

[Expression Language Principles](#)

[Expression Operators](#)

[Variables](#)

[Input Parameters](#)

[Functions](#)

[Lambda Functions](#)

[?](#) Variables

You can use several types of variables in the Expression Language syntax.

This page discusses:

- [Context Variables](#)
- [Local Variables](#)
- [Global Variables](#)

See Also

[Expression Language Principles](#)

[Types](#)

[Input Parameters](#)

[Functions](#)

[Lambda Functions](#)

Context Variables

Expressions are executed in a specific context, which in most cases, contains one or more variables. These variables are available for use in the expression.

In the User Interface, the **Expression editor** shows the list of variables (name and type) available for an expression.

Example:

```
person1 IN person2->friends
```

In this expression, `person1` and `person2` are context variables, and you can find if `person1` is present in the list of friends of `person2`.

Local Variables

It is possible to define local variables inside an expression. This allows you to reuse a value in different places within the expression.

Example:

```
now:DateTime = DateTime.now();  
{  
    month:now.month,  
    year: now.year  
}
```

In this expression, `now:DateTime` is a local variable to avoid repeating `DateTime.now()` several times. Without the local variable, the expression would be:

```
{  
    month:DateTime.now().month,  
    year: DateTime.now().year  
}
```

Global Variables

A global variable is a variable that is accessible to any graph processing operator in a graph. A global variable is declared in a graph and can be used in any expression within the graph.

To declare a global variable in a graph, add the `ToVariable` operator. You cannot use a global variable before it is declared.

Example: In a graph with a `ToVariable` operator that declares a variable named `average`, you can use the variable in any expression within the graph.

```
person.age > @average
```

Input Parameters

You can define input parameters (which can be values or functions) and use them anywhere in an operator expression, to inject values into the expression.

An input parameter starts with the `$` sign, followed by the name of the parameter.

For example, if you define a `limit` parameter that returns an `Integer` in your data query, the following expression uses the parameter:

```
person.age > $limit
```

If the placeholder has no arguments, parentheses are optional. The previous example is strictly equivalent to:

```
person.age > $limit()
```

If your parameter has arguments, the expression language interprets it as a function instead of a value. For example, if you define a `predicate` parameter that accepts an `Integer` as argument and returns a `Boolean` value, you can use the expression `$predicate(person.age)`.

When Data Perspective Studio calls the data query, the expression `"age > 10"` is valid for the `predicate` argument.

See Also

[Creating Input Parameters](#)

[Expression Language Principles](#)

[Types](#)

[Variables](#)

[Expression Operators](#)

[?](#) Expression Operators

You can use many operators in the Expression Language.

This page discusses:

- [Data Model Used in Examples](#)
- [Field Access](#)
- [Boolean Operation](#)
- [Conditional Expressions](#)
- [Comparison](#)
- [Number Manipulation](#)
- [Item Manipulation](#)
- [List Manipulation](#)
- [Map Manipulation](#)
- [Set Manipulation](#)
- [Tuple Manipulation](#)
- [Cast Operation](#)
- [Bitwise Operation](#)

See Also

[Expression Language Principles](#)

[Types](#)

[Variables](#)

[Input Parameters](#)

[Functions](#)

[Lambda Functions](#)

Data Model Used in Examples

The following subsections use examples based on this data model:

```
package app

class Issue {
    related : List<Reference<Issue>>
    title : Text
    description : Text
    reporter : Reference<Person>
    owner : Reference<Person>
}

class Person {
    firstName : String
    lastName : String
    rolesByContext : Map<List<Reference<Role>>>
}

class Role {
    title : Text
    description : Text
}

class Comment {
    issue : Reference<Issue>
    author : Reference<Person>
    publicationDate : DateTime
}
```

```
}
```

Field Access

Operator	Description	Sample
.	Fields (Tuple) or items	<code>p.price</code>
[]	Attributes (Item) and properties access	<code>map["key"], list[0]</code>
->	Use the arrow operator to specify a traverse reference from index unit items. Use <code>-></code> to specify a direct traverse.	<code>issue->owner.firstNameissu</code> Note: You can apply <code>-></code> on: <code>List</code> This operator returns an <code>Item</code> or a Warning: Even if the arrow op should not expect to get the sa
<-	The reverse arrow operator <code><-</code> always returns a <code>List<Item></code> , as it is impossible to specify the cardinality of the relation.	<code>issue<- [app.Comment.issue]</code> In this example, we can see that a reference a Ticket.

Boolean Operation

Operator	Description
NOT	Boolean NOT
AND	Boolean AND
OR	Boolean OR

Conditional Expressions

Operator	Description	Sample
<code>switch</code>	Similar to the If else operator. It is more readable when you have complex expressions.	<code>switch</code>
<code> </code>	Coalesce	<code>p.price</code>
<code>?:</code>	Ternary operator	<code>foo ==</code>

if	If else operator. Parentheses and curly bracket are mandatory."else" section is optional.	if (p.age
----	---	-----------

Comparison

Operator	Description	Sample
<= >= > <	Relational operators	p.age < 4 page.age IS Integer
==	Equal operator	
!=	Different operator	
IS	Use this operator to test the type or the nullity of an object.	p IS Integerp IS Item<social.Person>p IS null The IS operator does not apply implicit casting, which means that you cannot compare a string with an integer. However, IS supports the inheritance of Items, which means that you can compare a string with an integer if the integer is a subclass of the string.

Number Manipulation

Operator	Description	Sample
+ -	As unary operators	-2
* / %	Multiply, Divide	p.age*4 p.age/4
+ -	As binary operators	p.age + 4

Item Manipulation

Operator	Description	Sample
...	The spread operator expands an item into a tuple. It retrieves all the item attributes to create a tuple out of them.	To convert an item in tuple and a list of items. item.lastName} Warning: Using ...item is a common mistake. Try to filter or aggregate items.

List Manipulation

Operator	Description
<code>IN</code>	Contained in
<code>...</code>	The spread operator expands a list into another list. You typically use this operator to append an element to a list.
<code>+</code>	This operator allows you to merge 2 lists together, or adds an element to a list.

Map Manipulation

Operator	Description
<code>...</code>	The spread operator expands a map into another map. It allows you to add an element to the map. Notes: If there are several occurrences of the same key in the maps, it retains the value of the last key. Keys cannot be <code>NULL</code>. The expression language does not add <code>NULL</code> keys to the map.
<code>+</code>	This operator allows you to merge 2 maps together.
<code>.entries</code>	Returns the list of key/value pairs.

Note: You can also use several properties with the map type. For more information, see [Types](#).

Set Manipulation

Operator	Description
<code>+</code>	This operator allows you to merge 2 sets together, or adds an element to a set.

Tuple Manipulation

Operator	Description	Sam
<code>...</code>	The spread operator expands a tuple into another tuple. It allows you to add attributes to the tuple. Note: If there are several occurrences of the same attribute in the tuples, it retains the value of the last attribute.	<code>{ .</code> Yo
<code>+</code>	This operator allows you to merge 2 tuples together. Note: If there are several occurrences of the same attribute in the tuples, it retains the value of the last attribute. For example, <code>{name: "Bob", age: 20} + {name: "Alice", height: 180}</code> returns <code>{name: "Alice", age: 20, height: 180}</code>	<code>{n</code> att

Cast Operation

Operator	Description	Sample
AS	Allows you to cast an element to a compatible type.	p AS DateTime p

Bitwise Operation

Operator	Description
<< >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR

Functions

The Expression Language has a **Catalog** containing many functions. When calling a function, you can optionally use the names of the f

For example, the following function calls are all equivalent:

```
Date.of(2016, 1, 31)
Date.of(year: 2016, month: 1, day: 31)
Date.of(day: 31, month: 1, year: 2016)
Date.of(2016, 1, day: 31)
```

You can call some functions directly on a variable. These syntaxes are equivalent:

```
values : List<Integer> = [1,3,5,7];
values.filter(value => value % 3 == 0)
List.filter(values, value => value % 3)
```

To see which functions you can call directly, use the autocomplete on a variable. For more information about

See Also

[Expression Language Principles](#)

[Types](#)

[Variables](#)

[Input Parameters](#)

[Expression Operators](#)

[?](#) Lambda Functions

You can use lambda functions in the Expression Language.

The general syntax of a lambda function is:

```
(parameterName1:parameterType1, parameterName2:parameterType2, ...) => expression
```

When there is only one parameter, you can omit the parentheses. When the Expression Language can infer

Examples:

```
foo => foo + 2  
(foo:Integer, bar:Integer) => (foo * bar)  
(foo:Integer, bar) => foo + bar
```

You can use a lambda function as a parameter of a function. The parameter type must be `Function`. For more

```
values : List<String> = ["foo", "bar"];  
values.anyMatch(value => value == "bar")
```

See Also

[Expression Language Principles](#)

[Variables](#)

[Input Parameters](#)

[Expression Operators](#)

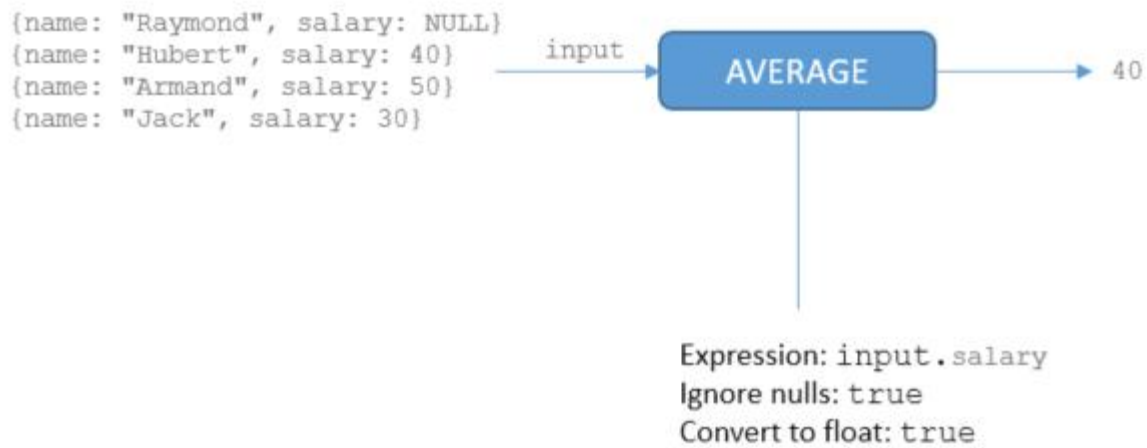
[Functions](#)

[?](#) AVERAGE

The **AVERAGE** aggregation function calculates an average value from an input stream.
For example, you can find the average income of a population.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your input stream.

Expression	Extracts the numerical value of each element of the input stream. For example, <code>person.income</code> if your input is a <code>Person</code> . Default: <code>identity function</code> .
Ignore nulls	By default the aggregation ignores null elements. If <code>false</code> , the aggregation returns <code>null</code> if it finds at least one <code>null</code> . Default: <code>true</code> .
Convert to float	By default the aggregation keeps the type of the numerical element. For example, if the income is an <code>Integer</code> , the result is an <code>Integer</code> . Default <code>false</code> .



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[?](#) COMPOSITE

The **COMPOSITE** aggregation function allows you to combine existing aggregations, including other composite aggregations, together.

This page discusses:

- [Configuration](#)
- [Example: Calculate the Body Mass Index](#)

Configuration

Parameter	Description
Expression	The expression can only combine existing aggregations together.

Example: Calculate the Body Mass Index

In this example, you want to calculate the average body mass index (BMI) of a population. The BMI is defined as the weight in kilograms divided by the square of the height in meters.

To make this calculation, you can use a composite aggregation that combines an aggregation for the average

average_weight

Average

—

average_height

Average

—

body_mass_index

Composite

—

Expression

average_weight / (average_height*average_height)

See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

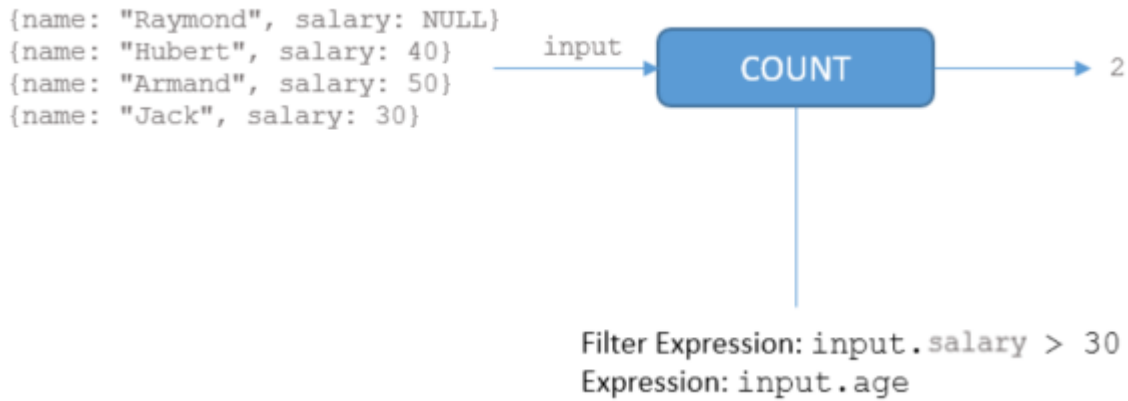
[Choose Where to Apply Aggregations](#)

[?](#) COUNT

The **COUNT** aggregation function calculates the number of documents in the input stream.

For example, you can count the number of employees.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of you



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[?](#) CUSTOM

The **CUSTOM** aggregation function takes a stream as input and returns a single value. The value type depends on the configuration.

This page discusses:

- [Configuration](#)
- [Input](#)
- [Output](#)
- [Example: Computing a Standard Deviation](#)

Configuration

This aggregator follows 3 phases:

1. It initializes variables.
2. It iterates over the stream while updating the variables for each element.
3. It returns an output expression value using the final values of the variables.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your
Variables	This aggregator uses n variables. These variables are updated through expressions that are executed fo
Updates	Defines how to iterate with Variables on specific objects.
Output expression	When the input stream is fully consumed, the output expression creates the single value that goes out of This expression can access all variables. By default, this operator operates in sequential mode, keeping the element order from the stream.
Terminal condition	You can use a terminal condition to stop the iteration on the stream (phase 2) and directly go to the ou

Input

There is one Input. There are no constraints on the element type or on the number of elements in the stream.

Output

There must be one Output. As an aggregator, this function emits a single value. The type of this value depends on the output expression.

Example: Computing a Standard Deviation

Let us assume that the average of the values of the stream is already computed and stored in a global `@avg`.

In this example, the input stream is a stream of Integer.

1. Declare 2 variables:

```
2. count, type : Integer, default value : 0
```

```
    sum_squared_diff, type : Float, default value : 0.0
```

3. Update the 2 variables:

```
4. count = count + 1
```

```
    sum_squared_diff = sum_squared_diff + Math.pow(val-@avg, 2)
```

Where: `val` is the element coming from the stream.

5. Declare the output expression:

```
    sum_squared_diff / count
```

See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[!\[\]\(b803c33e199dbe3d036f66930e9986e2_img.jpg\)](#) **GRAPH**

The **GRAPH** aggregation function uses a subgraph to define how to process the aggregation. It is an advanced function made to specify

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of
Processing subgraph	Defines a subgraph. It is called once and must have one graph Input and one graph Graph Output. The graph Input receives all the input elements on which to process the aggregation. The Graph Output is bound to the aggregation output. Note: You can also configure Subprocessing graphs in the MAP operator. For more information, s

See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[!\[\]\(c3a987993e45f8adbe3dcf2d4cc1b19a_img.jpg\)](#) **LINEAR**

The **LINEAR** aggregation function can take several time series as input, and merge them into a single time series in output.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your input stream
Expression	Extracts a time series from the input element. For example, <code>income.history</code> if your input is a stream of persons. Default: <code>identity function</code>.

Interpolation	Specify how to draw missing values between 2 known points in the time series. Linear: Estimates the missing value by joining the 2 known data points with a line. Previous: Takes the same value as the previous known data point in the time series to draw the missing value. Next: Takes the same value as the next known data point in the time series to draw the missing value. Nearest: Takes the same value as the nearest known data point in the time series to draw the missing value.
Function	Choose between Average, Max, Min, Percentile, Sum. For example, if you choose Average, for each time instant, the aggregation makes an average of each point in the time series.

See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

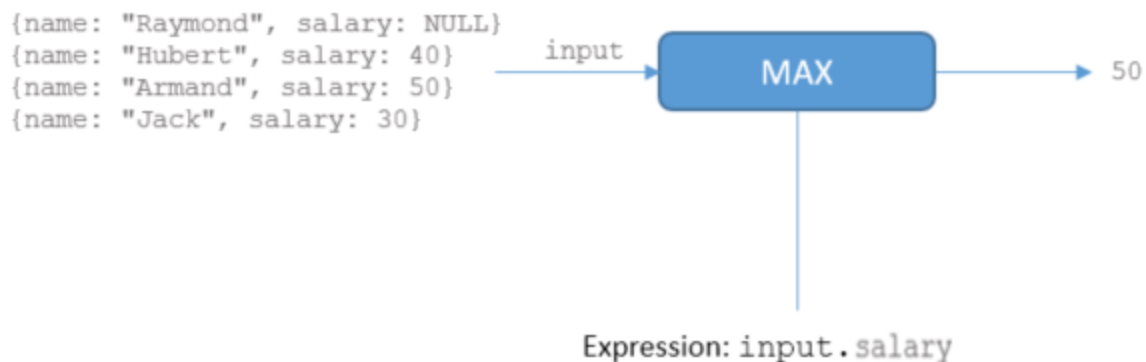
[Choose Where to Apply Aggregations](#)

[?](#) MAX

The **MAX** aggregation function calculates the max value from an input stream.

For example, you can find the highest income among a stream of persons.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your input stream.
Expression	Extracts the numerical value of each element of the input stream. For example, <code>person.income</code> if your input stream contains persons. Default: <code>identity function</code> .
Ignore nulls	By default the aggregation ignores null elements. If <code>false</code> , the aggregation returns <code>null</code> if it finds at least one null element. Default: <code>true</code> .



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

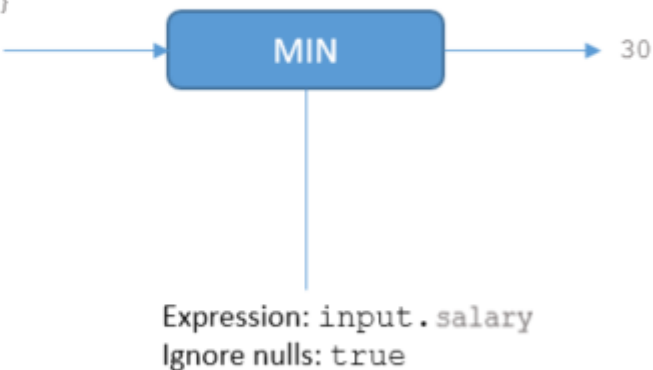
[?](#) MIN

The **MIN** aggregation function calculates the min value from an input stream.

For example, you can find the lowest income among a stream of persons.

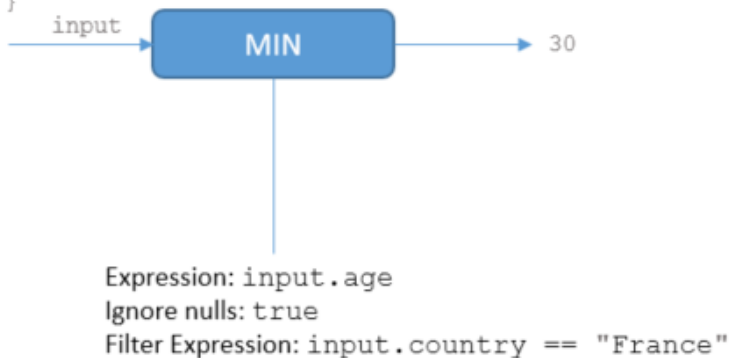
Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your input stream.
Expression	Extracts the numerical value of each element of the input stream. For example, <code>person.income</code> if your input stream contains objects of type <code>Person</code> . Default: <code>identity function</code> .
Ignore nulls	By default the aggregation ignores null elements. If <code>false</code> , the aggregation returns <code>null</code> if it finds at least one null element. Default: <code>true</code> .

```
{name: "Raymond", salary: NULL}
{name: "Hubert", salary: 40}
{name: "Armand", salary: 50}
{name: "Jack", salary: 30}
```



Min with Filter

```
{name: "Raymond", age: NULL, country : "France" }
{name: "Hubert", age: 30, country : "France"}
{name: "Armand", age: 50, country : "France"}
{name: "Jack", age: 20, country : "USA "}
```



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[?](#) PERCENTILE

The **PERCENTILE** aggregation function calculates a configured number of percentiles from numerical values, based on each input element.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your input.

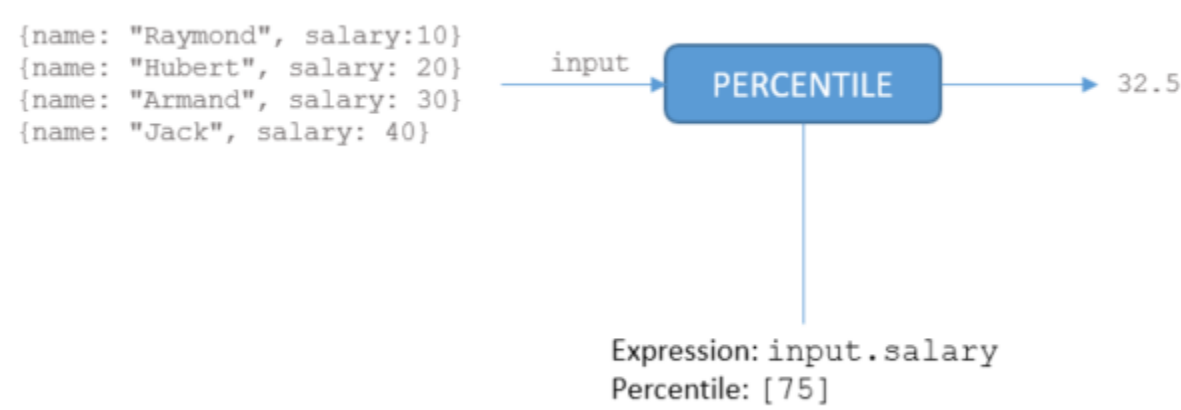
Expression	Extracts the numerical value of each element of the input stream. For example, <code>person.income</code> if your i Default: <code>identity function</code> .
Percentiles	Specify the number of percentiles. For example, <code>[75]</code>

The output contains all defined percentile values.

To be more precise, the output type is a

`List<Tuple<percentile: Integer, value: Float>>` with `percentile ∈ \[1 , 99\]`

The number of elements in the output list corresponds to the number of percentiles defined in the operator



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

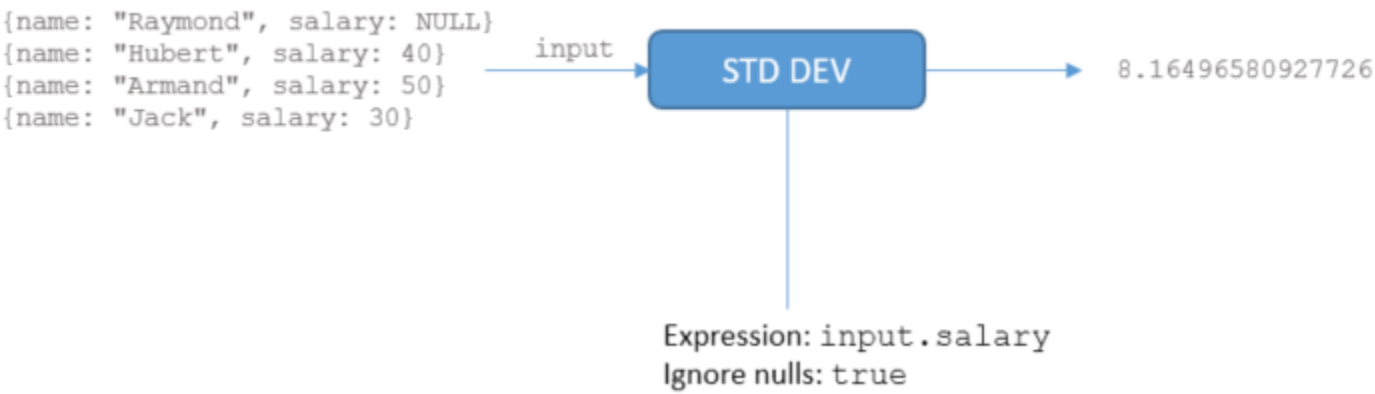
[Choose Where to Apply Aggregations](#)

STANDARD DEVIATION

The **STANDARD DEVIATION** aggregation function calculates the standard deviation value from an input stream. For example, you can find the standard deviation of incomes among a stream of persons.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your in

Expression	Extracts the numerical value of each element of the input stream. For example, <code>person.income</code> if your i Default: <code>identity function</code> .
Ignore nulls	By default the aggregation ignores null elements. If <code>false</code> , the aggregation returns <code>null</code> if it finds at lea Default: <code>true</code> .



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

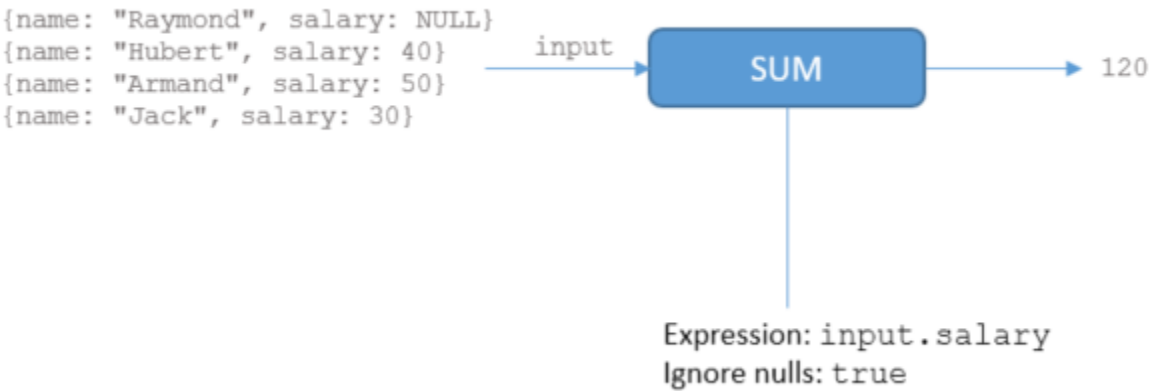
[Choose Where to Apply Aggregations](#)

[SUM](#)

The **SUM** aggregation function calculates the sum of numerical values over an input stream.
For example, you can process the total sales.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your in
Expression	Extracts the numerical value of each element of the input stream. For example, <code>person.income</code> if your i Default: <code>identity function</code> .
Ignore nulls	By default the aggregation ignores null elements. If <code>false</code> , the aggregation returns <code>null</code> if it finds at lea

	Default: <code>true</code> .
--	------------------------------



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[TAKE FIRST](#)

The **TAKE FIRST** aggregation function returns the first element of the input stream.
For example, you can pick the oldest person among a stream of persons.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your input stream.
Expression	Extracts the numerical value of each element of the input stream. For example, <code>person.income</code> if your input stream contains person objects. Default: <code>identity function</code> .
Sorter	Sorts elements before taking the first.



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

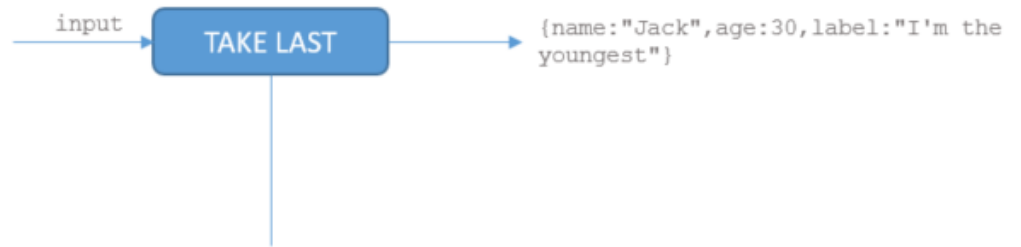
[Choose Where to Apply Aggregations](#)

[TAKE LAST](#)

The **TAKE LAST** aggregation function returns the last element of the input stream.
For example, you can pick the youngest person among a stream of persons.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your input stream.
Expression	Extracts the numerical value of each element of the input stream. For example, <code>person.income</code> if your input stream contains person objects. Default: <code>identity function</code> .
Sorter	Sorts elements before taking the last.

```
{name: "Hubert", age: 60}
{name: "Armand", age: 50}
{name: "Jack", age: 30}
```



Expression: `{...input, label: "I'm the youngest"}`

See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

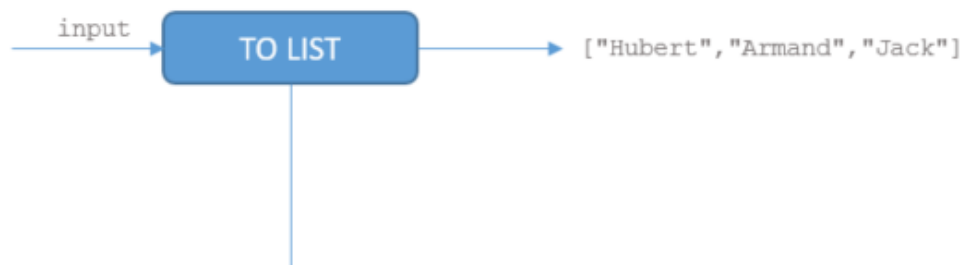
[?](#) TO LIST

The **TO LIST** aggregation function returns a single list of all documents from the input stream.

For example, you can aggregate a stream of persons to a single list of full names.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of you
Expression	An expression extracting the element to put into the <code>Set</code> . For example, <code>person.fullname</code> .

```
{name: "Hubert", age: 60}
{name: "Armand", age: 50}
{name: "Jack", age: 30}
```



Expression: `input.name`

See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[?](#) TO MAP

The **TO MAP** aggregation function aggregates an input stream to a Map.

For example, you can aggregate a stream of persons to a `Map<String, Integer>` with the full name as key and the age as value.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your input.
Key expression	An expression extracting the key element for the Map. For example, <code>person.fullname</code> .
Value expression	An expression extracting the value element for the map. For example: <code>person.age</code> .
Multivalued	If <code>true</code> , the value element is a <code>List</code> . Default: <code>false</code> .

```
{name: "Hubert", age: 60}
{name: "Armand", age: 30}
{name: "Jack", age: 30}
```



```
Map(
  <30, ["Armand", "Jack"]>,
  <60, ["Hubert"]>)
```

Key expression: `input.age`
Value expression: `input.name`
Multivalued: `true`

See Also

[RESULT SET](#)

[AGGREGATE](#)

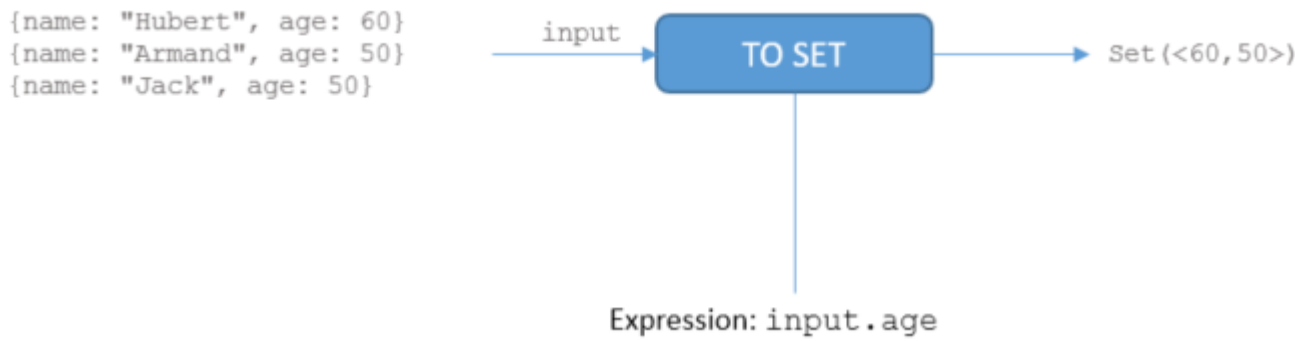
[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[?](#) TO SET

The **TO SET** aggregation function aggregates an input stream to a *Set*.
For example, you can aggregate a stream of persons to a *Set<String>* retrieving their ages.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of you
Expression	An expression extracting the element to put into the <i>Set</i> . For example, <i>person.age</i> .



See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[?](#) TO TIME SERIES

The **TO TIME SERIES** aggregation function takes a list of time instants and values to create a time series.

Parameter	Description
Filter expression	An expression to keep some elements only. It is useful to process the aggregation on a subpart of your data.
Value	Enter an expression to extract the value.
Instant	Enter an expression to extract the time instant.
Function	The aggregation uses this function when there are duplicates for a given time instant. Choose between Average , Max , Min , Percentile , Sum .

See Also

[RESULT SET](#)

[AGGREGATE](#)

[GROUP BY](#)

[Choose Where to Apply Aggregations](#)

[?](#) Partitioners

This section presents the partitioning functions, also called **Partitioners**, available in the **GROUP BY** operator that you can add to your queries. Use partitioners to split a dataset in groups. For example, in a dataset of persons, you can group persons having the same age, group persons by gender, etc.

In this section:

- [Date](#)
- [Predicate](#)
- [Range](#)
- [Value](#)
- [ValueSet](#)

[?](#) Date

In the **GROUP BY** operator, the **Date** partitioning function allows you to create date-based partitions.

Parameter	Description
Enable year field	Creates partitions by year. Default value: <code>true</code> .
Enable quarter field	Creates partitions by quarter (from 1 to 4). Default value: <code>false</code> .
Enable month field	Creates partitions by month (from 1 to 12). Default value: <code>true</code> .
Enable day field	Creates partitions by month day (from 1 to 31). Default value: <code>true</code> .
Enable week year field	Creates partitions by week according to ISO. Unlike Enable week field , it keeps 7 days per week. Default value:
Enable week field	Creates partitions by week (from 1 to 53). Be careful, with this mode, the first and last weeks of the year may
Enable week day field	Creates partitions by week day (from 1 to 7). Default value: <code>false</code> .
Enable hour field	Creates partitions by hour. Default value: <code>false</code> .
Enable minute field	Creates partitions by minute. Default value: <code>false</code> .
Enable second field	Creates partitions by second. Default value: <code>false</code> .
Max groups	An expression returning an Integer. When defined, fields are disabled automatically (starting from the second
Fill gaps	Adds empty partitions to keep a continuous partition sequence. Default value: <code>false</code> .

See Also

[GROUP BY](#)

[Expression Language](#)

[Defining Data Queries](#)

[?](#) Predicate

In the **GROUPBY** operator, the **Predicate** partitioning function allows you to partition data according to a list of predicate groups. An element belongs to a partition if the corresponding predicate is `true`.

Parameter	Description
Matching policy	Specifies how to manage elements matching several predicates. First Only: Only the first partition matching the element gets it. All: Elements can be in several partitions. Default: First Only .
Groups	The list of predicate . For example, <code>person.age < 20, person.age > 60</code>

See Also

[GROUP BY](#)

[Expression Language](#)

[Defining Data Queries](#)

Range

In the **GROUPBY** operator, the **Range** partitioning function allows you to partition data by ranges. There are 4 ways to define these ranges. This page discusses:

- [Explicit Ranges](#)
- [Equivalent Sized Ranges](#)
- [Fixed Sized Ranges](#)
- [Percentile Ranges](#)

Explicit Ranges

Use the **Explicit** range definition to define each possible range manually.

For example, you can use it to create 3 groups of persons according to their age:

- `[0, 20[`
- `[20, 62]`
- `]62, +infinity[`

Equivalent Sized Ranges

Use **Equi Sized** range definition to create ranges having the same width.

For example, you can use it to create 3 groups of persons according to their age automatically.

If the youngest one is 20 years old and the oldest one 43 years old, the **Equi Sized** range definition creates

Fixed Sized Ranges

Use **Fixed Sized** range definition to create ranges with a specified width.

For example, you can use it to create one group per 10 years automatically.

If the youngest one is 20 years old and the oldest one 43 years old, the **Fixed Sized** range definition creates

Percentile Ranges

Use **Percentile** range definition to create ranges based on percentiles.

For example, you can use it to create one partition of 10% of the youngest persons and 10% of the oldest ones.

Include the list of ranges between 0 and 100.

See Also

[GROUP BY](#)

[Expression Language](#)

[Defining Data Queries](#)

Value

In the **GROUP BY** operator, the **Value** partitioning function allows you to partition elements according to a grouping key.

If the grouping key is a list, the **Collection policy** specifies how to handle it with one of the following values.

Value	Description
List (default value)	The list is used as a grouping key entirely. For example, one group for ["a", "b"], another for one ["a", "c"]
Each Element	Each item of the list is used as a grouping key separately. For example, an element with ["a", "b", "c"] v
Each Sublist	Each prefix sublist of the list is used as a grouping key separately. For example, an element with ["a", "b", "c"]

See Also

[GROUP BY](#)

[Expression Language](#)

[Defining Data Queries](#)

[?](#) ValueSet

In the **GROUP BY** operator, the **ValueSet** partitioning function is similar to the **Value** partitioning function, except that it creates groups

For example, to create one group of persons called "Jean" or "Michel", and another group of persons called

For more information about the **Collection policy** attribute, see [Value](#).

See Also

[GROUP BY](#)

[Expression Language](#)

[Defining Data Queries](#)

[?](#) Data Query Tips and Tricks

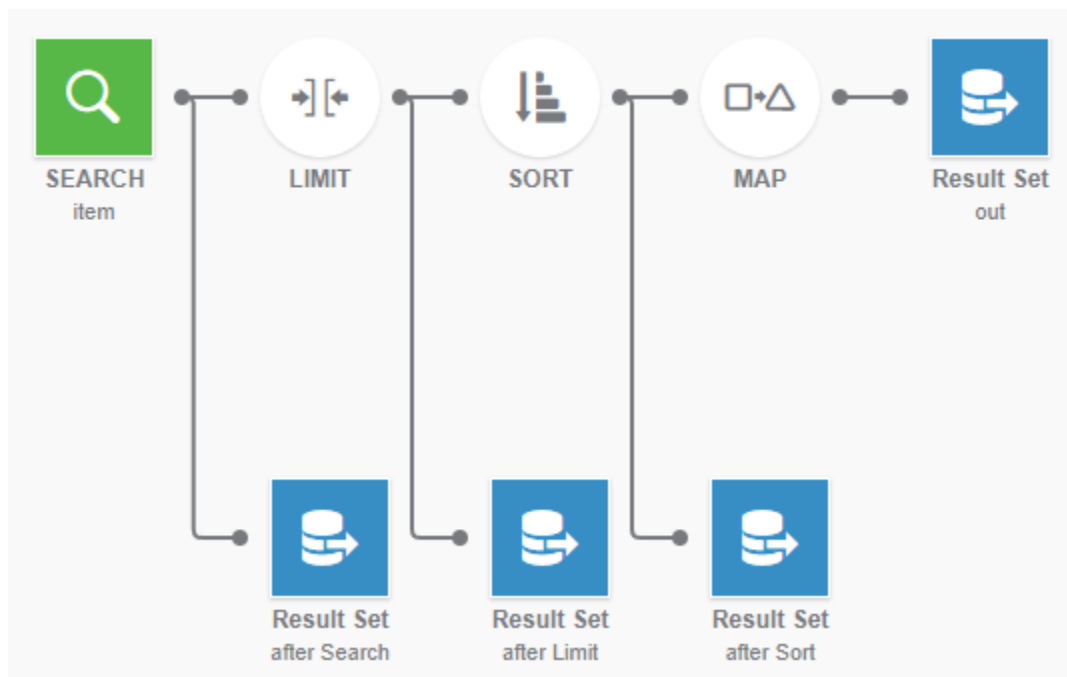
This page gives several tips and recommendations to configure your [?](#) data queries properly.

This page discusses:

- [See Intermediate Results in a Data Query](#)
- [Choose Where to Apply Aggregations](#)
- [Aggregate Several Query Results](#)
- [Use Item Instead of Tuples](#)
- [Transform a Nested List into a Flat List](#)

See Intermediate Results in a Data Query

When creating a [?](#) data query, the good practice is to put a [RESULT SET](#) operator after each operator. Verifi



Choose Where to Apply Aggregations

You can make aggregations either at the query level or the visualization level.

Level	Description
Aggregation at the query level	In a data query or a data fusion, you can make aggregations in the following operators: AGGREGATE and GROUP BY transformation operators: You cannot modify these aggregations. - These operators can make your query more complex if you want to perform several aggregations. - You cannot reuse these aggregations at the visualization level. Recommendation: Avoid adding multiple AGGREGATE and GROUP BY operators.
	RESULT SET terminal operator: You can define several aggregations in this operator at the query level. This is very useful to declare several aggregations once and use them for several visualizations. You can also define complex aggregations that are not available at the visualization level (TOPI, etc.). Recommendation: Choose this method to mutualize your aggregation definitions.
Aggregation at the visualization level	In several visualizations (from Properties ? > Data > Measures) you can: - Select one of the aggregations previously defined in the RESULT SET called by the visualization. - Define an aggregation locally. This is useful if the aggregation is specific to the visualization. Tip: To use a same aggregation in several visualizations, it is better to declare it at the RESULT SET level.

Aggregate Several Query Results

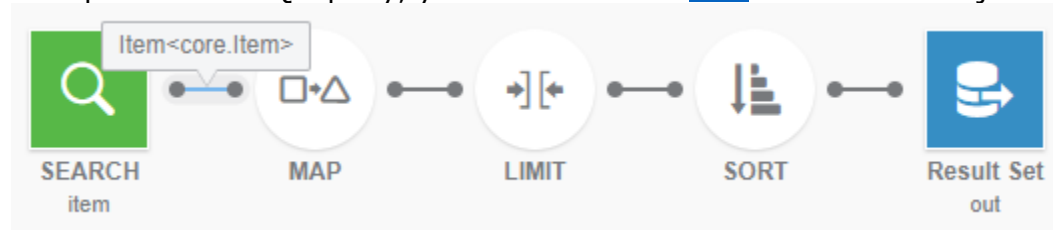
If you have several queries that are producing similar results, and want to get a single result aggregating the

Use Item Instead of Tuples

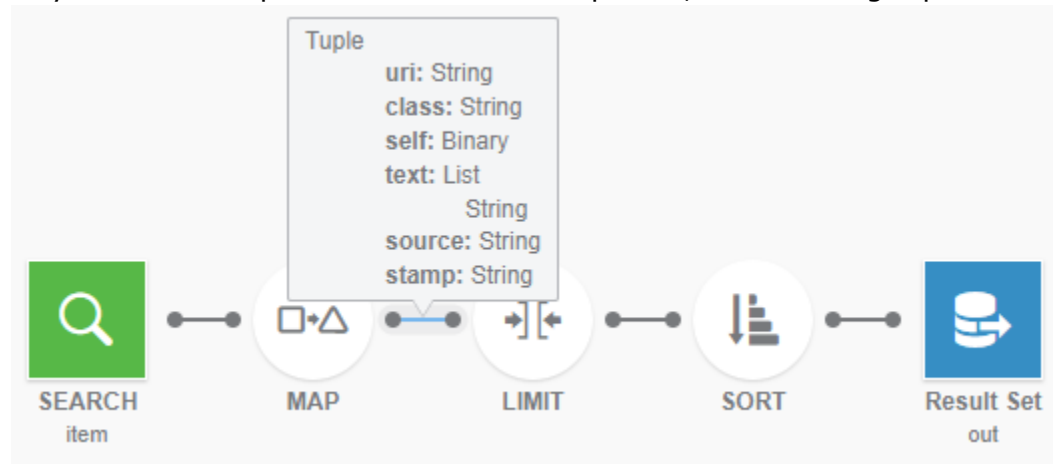
With the Semantic Graph Index, it is better to manipulate items rather than tuples, for performance reasons and the number of attributes.

The following use case shows how you would proceed in SQL, and how to do it more efficiently in Semantic

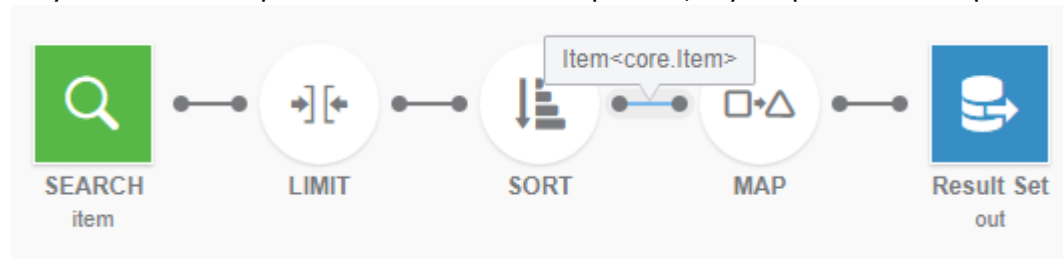
To reproduce an SQL query, you would add the [MAP](#) with its item objects at the beginning of the query.



In your Data Perspective Studio [?](#) data queries, avoid adding tuple transformation as in SQL. In SQL, you could



In your Data Perspective Studio [?](#) data queries, try to put the [MAP](#) operator and object items at the end of the



Transform a Nested List into a Flat List

You can make a flat list out of a nested list using the `List.flatMap()` function.

It transforms a `List<List<E>>` into a `List<E>`. For example:

```
[[1, 2], [3]].flatMap(list => list)  
returns [1, 2, 3].
```

See Also

[Aggregation Functions](#)

[Defining Data Queries](#)

[Use Cases](#)

[Frequently Asked Questions](#)

[?](#) Frequently Asked Questions

This page answers several questions that you may have concerning Data Perspective Studio.

This page discusses:

- [Retrieve User and Tenant Information](#)
- [Can I Filter Data?](#)
- [Can I Search for Text?](#)
- [Can I Quickly Generate Data for Demo?](#)

Retrieve User and Tenant Information

The **Catalog** available in the **Expression Editor** contains several functions to apply security or filter documents.

Function	Description
<code>User.login()</code>	Returns the login of the connected user.
<code>User.uuid()</code>	Returns the <code>uuid</code> of the connected user. Use this function in RDF in
<code>User.locale()</code>	Returns the locale of the connected user, or <code>en</code> (English) if unspecified
<code>Environment.tenant()</code>	Returns the current tenant.

Can I Filter Data?

You can filter out chart data on all dimensions and fields. There is no need to configure a faceting behavior.

You can also add the **Active Filters List** and **Refine Panel** visualizations. You select the dimensions on which

Limitation: For the moment, we do not support hierarchical facets.

Can I Search for Text?

Searching for text in the index is a very common use case, that you can configure using the `Text.hasTerm()` operator.

Can I Quickly Generate Data for Demo?

You can use the **GENERATOR** operator when you want to quickly generate a dataset to feed your data queries.

See Also

[About Expressions in Operations](#)

[Data Query Tips and Tricks](#)

[Use Cases](#)

[?](#) Column Chart

You can use the  **Column** chart visualization to present categorical data with rectangular bars.

You can use either vertical or horizontal bars (by switching to **Bar** chart) to make numerical comparisons across categories.

Usage guidelines:

- Whenever you can, provide controls that let users filter and sort large datasets. This allows them to refine the view to better suit their needs.
- Differentiate between histograms and bar charts. Histograms help viewers see distributions, while bar charts help viewers compare values across categories.
- Color is not automatically applied to bar charts by default. Use colors to highlight specific columns for storytelling.

This task shows you how to:

- [Define the Data to Show](#)
- [Switch to Another Type of Chart](#)
- [Define the Dimensions](#)
- [Define the Measures](#)
- [Define the Value Colors](#)
- [Define the Value Colors in Breakdown](#)
- [Drill-Down and Drill-Up on Dimensions](#)
- [Add Time Series to the Chart](#)
- [Define Display Options for the Axes](#)
- [Split a Chart with Multiple Measures](#)
- [Split a Chart by Dimension Categories](#)
- [Show Underlying Data in a Data Panel](#)
- [Reduce the Scope of Dimensions with Local Filters](#)
- [Configure the Display](#)
- [Configure the Chart Legend](#)
- [Add Effects On Chart Click](#)
- [Add Effects \(On Apply selection click\) for Multiple Selection](#)
- [Configure the Actions Menu](#)
- [Add Target Lines to the Chart](#)
- [Add Areas to Chart Targets](#)

- [Add Marker Points to the Chart](#)

See Also

Define the Data to Show

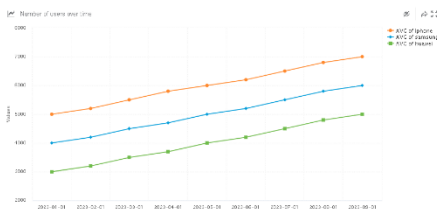
1. In the **Properties** panel, go to **Data**.
2. Select a **Query**, that is either a **data query** or a **data fusion**, previously defined in the **Queries**
3. If you have selected a data query, also select a **Result Set**.

Switch to Another Type of Chart

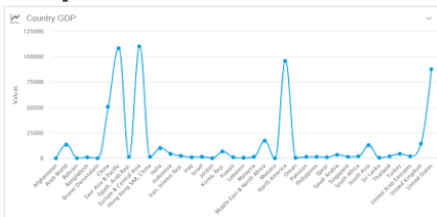
You can switch from one type of chart to another in one click.

1. In the **Properties** panel, go to **Data**.
2. In the **Type** section, click the icon corresponding to the type of chart that you want to show.

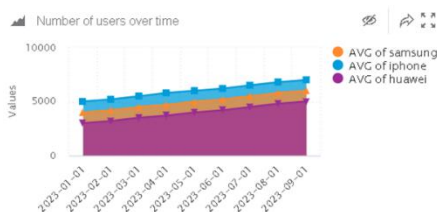
Line chart



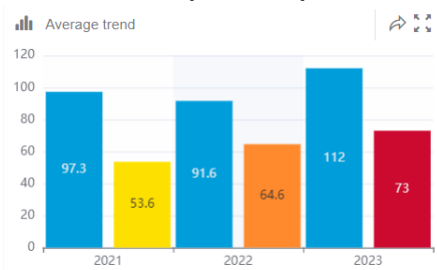
Spline chart



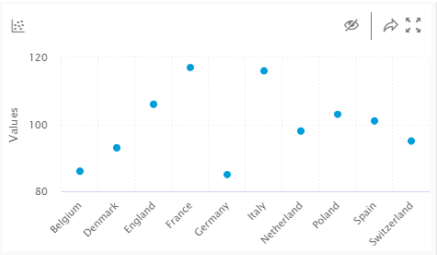
Area chart



Bar chart (vertical)



Point chart



Define the Dimensions

You must configure the dimension to show in the chart by selecting a field.

Data Perspective Studio suggests the dimensions available in the **Result Set** of the selected  data query, or

1. In the **Properties** panel, go to  **Data**.
2. In the **Axis** section, click  to add a **Dimension**.
3. For **Field**, select the field to use as dimension.
4. **Optional:** If your field stores data of type `List`, you can select **Show as hierarchical tree** to get a display

The chart first shows the values of the first level of hierarchy in the dimension. On click, the chart shows

A breadcrumb allows you to return to a previous level as in a standard multidimension drill down.

5. **Optional:** Click **More options** to open the advanced configuration properties.

Option	Description
Visible values	Select the initial number of categories to display for the dimension.
Limit	Select the maximum number of categories to display for the dimension. When the number of categories crosses this limit, the chart creates a pagination gathering all the remaining
Group remaining values	If the number of categories crosses the defined Limit , this option groups all the remaining categories that When you select this option, the chart has no pagination. When you click the Others category, Data Perspective Studio filters the current view to exclude all the other Note: This option does not work for charts with breakdown.
Sort by	Select how to sort dimension categories. Alphanum: sorts the categories alphanumerically. Custom: lets you specify how to sort the categories contained in the dimension by defining Ordered values Measure: lets you select any aggregation defined either in  Data > Values > Measures or in the RESULT SET Click  or  to sort in ascending or descending order.

Aggregation	Select the type of aggregation to show in the sector. The default is COUNT .
Format	You can select a text format: No formatting , lower case , UPPER CASE , Capital on first letter + no change (capitalizes the first letter of the first word and changes the rest to lowercase), Capital On All First Letters (capitalizes the first letter of each word).

6. **Optional:** When you select a value in the chart, the **Show filtered out values** option shows the values that were filtered out.

Define the Measures

You can configure the measures to show for the Y axis. When there are two dimensions, Data Perspective S

1. In the **Properties** panel, go to  **Data**.
2. In the **Values** section, click  to add a **Measure**.
3. For **Value**, select the aggregation value to use in the Y axis.
4. **Optional:** Configure the display of chart series.

In **More options > Series display**, you can configure the following options depending on the selected t

Option	Description
Hide measure	You can hide a measure series from the chart. In that case, the measure tooltip information is still visible.
Type	You can switch to another type of chart.
Stack with measure	You can select another measure (also added in the Values section) and stack it with the current measure. Note: You can stack 2 measures only, either if they do not have any breakdown, or if they share the same breakdown.
Show stacked bar total value	Shows the total value at the top of each stacked Bar or Column chart. You can then select the Stacked bar label format , for example, a currency.
Line Style	Select the style of the chart line between Dashed , Dot , or Solid .
Line width	Select the width (in pixels) of the chart line.
Show outline blur	Surrounds the chart line with a light blur when series overlap with one another. It is typically used for line charts.
Point Symbol	Select the symbol of the chart points between Circle , Empty circle , Rectangle , Round rectangle , Triangle , Empty triangle , Star , Empty star , Hexagon , Empty hexagon , Circle with cross , Empty circle with cross , Triangle with cross , Empty triangle with cross , Star with cross , Empty star with cross , Hexagon with cross , Empty hexagon with cross .
Point size	Select the size (in pixels) of the chart points.

Connect missing points	Allows you to fill the chart series and get a continuous line when values are missing.
------------------------	--

5. **Optional:** For  **Line** charts, you can configure conditional formatting to highlight segments of a number. In **More options > Conditional formatting**, configure line segments for the chart dimensions.

Important: The order of the conditions defines their priority.	
Option	Description
Dimensions	Select the dimension on which you want to add one or several line segments.
Line segments	Add one or more line segments to the selected dimension.
Line segment	Configure the line segment using the following options: • Greater than dimension value: The line segment starts after the specified numeric value or date. • Override line style: Select whether to display a solid, a dot, or a dashed line from the segment. • Override line width: Select the width (in pixels) of the line segment.

6. **Optional:** Configure the chart data label in **More options**. In **More options > Data labels**, configure the display of data labels.

Option	Description
Data labels	Select the label position on the chart: Hidden: No data label appears. Inside: The data label appears inside the visualization element. For example, inside bars for bar charts. Outside: The data label appears outside of the visualization element. For example, on top of columns.
Data label values	Select how to show measure values in labels. It can be a numerical value, a percentage, or both.
Data label values in tooltip	Select how to show measure values in tooltips. It can be a numerical value, a percentage, or both.
Display on	Select the axis on which you want to show the measure.

7. **Optional:** Configure the chart tooltip and legend in **More options**. In **More options > Tooltip and legend** and in configure the label and format to use in the chart tooltip. **Note:** In **More options > Legend**, you can also use options to customize the chart legend only.

Customize tooltip	Select this option to define the tooltip that displays when you hover over values in the legend. Note: The options are similar to the Customize label options.
-------------------	---

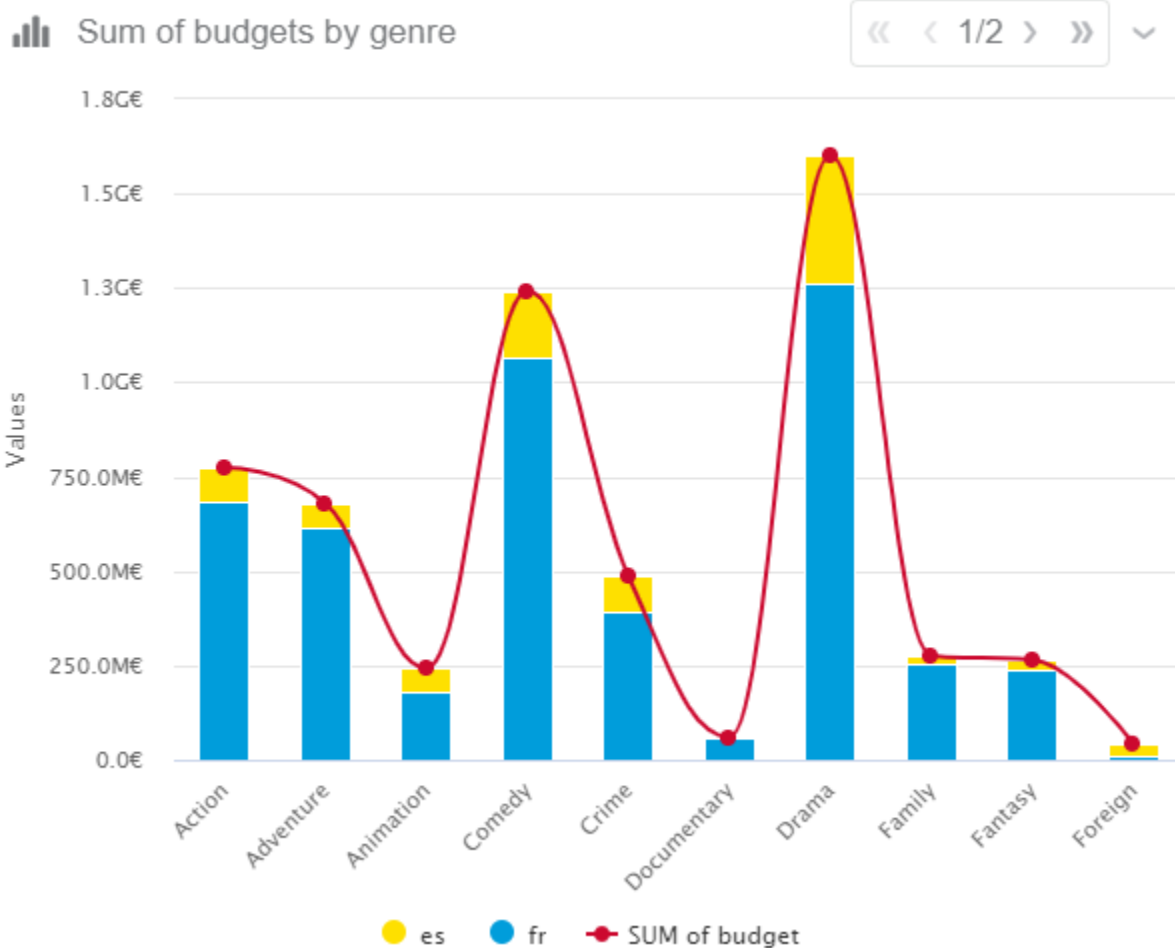
8. **Optional:** To create several series for the selected measure, select the **Breakdown** option. From **Field**, select another dimension to split the values.

The screenshot shows the 'Measures' configuration panel. At the top, it says 'Measures' with an information icon and a plus sign. Below, there's a section for 'Value' with a dropdown menu currently showing 'isOpen'. Underneath, the 'Breakdown' checkbox is checked, and the 'Breakdown by ...' section shows a dropdown menu with 'priority' selected. At the bottom of the panel, there are two 'More options' links.

Use case: Break down movie budgets by country.

In this example:

- The **Axis** shows movies by genre (Action, Adventure, ...)
- The **Measures**, have 2 SUM aggregations:
 - One **SUM** shows the budgets by genre in stacked bars, with a breakdown on country (France in ...)
 - Another **SUM** shows a red line for global budgets by genre.



Define the Value Colors

You can define the colors to apply depending on the values of a selected field.

1. In the **Properties** panel, go to **Data**.
2. In the **Colors** section, select the colors to apply to the field values.

Option	Description
Type	Select how you want to configure the color mapping, that is By dimension or By measure .
Mapping	Specify how to map field values with colors: • Manual: To select one or more colors manually. This is the default color mode associated with a visualization. • Global Palette: To use the global color palette that is common to all visualizations. It associates field values with colors. To specify several colors, select Customize.

	To override the automatic color mapping, select Customize. Important: The Global Palette colors are common to all visualizations in your app for consistency. This means that to have the same color on two charts, the values must come from fields with the same data type. • Use display rule: To use the color mapping defined in ? Resources > ? Display rules. Note: There is a limit in the number of colors available. Data Perspective Studio reuses the same colors when the limit is reached.
Customize	If the Type is By dimension, you can use this option, you can search for values and customize their colors. There is a limit to this customization, which blocks the option when there are more than 100 values. Important: The colors in Customize apply to the current visualization only.
Additional mapping	If the Type is By dimension, you can use this option to define conditions to apply colors. To do so, you must define a condition. For example, you can define to associate a color when the value is inferior (<) to the expected value. matches: allows you to specify a regular expression to choose the color or the icon to apply, depending on the value. Important: What you define in Additional mapping has precedence over what you define in Mapping.
Splitting strategy	If the Type is By measure, you can use this option to split the color scale into intervals. There are 2 splitting strategies: Gradient: Split the color scale into N gradient colors. Discrete: Split the color scale into N discrete intermediate colors.

3. **Note:** There are priorities if you define several colors for the same field value. Manual customizations have priority over display rules.

Define the Value Colors in Breakdown

When using the **Breakdown** option in **Measures**, you can define the colors to apply depending on the values of the dimension.

1. Click **More options**.
2. In the **Colors** section, select the colors to apply to the field values.

Option	Description
Type	Select how you want to configure the color mapping, that is By dimension or By measure .
Mapping	Specify how to map field values with colors: • Manual: To select one or more colors manually. This is the default color mode associated with a field. To specify several colors, select Customize. • Global Palette: To use the global color palette that is common to all visualizations. It associates field values with colors from the global palette.

	To override the automatic color mapping, select Customize. Important: The Global Palette colors are common to all visualizations in your app for consistency. This means that to have the same color on two charts, the values must come from fields with the same name. • Use display rule: To use the color mapping defined in ? Resources > ? Display rules. Note: There is a limit in the number of colors available. Data Perspective Studio reuses the same colors when the limit is reached.
Customize	If the Type is By dimension, you can use this option, you can search for values and customize their colors. There is a limit to this customization, which blocks the option when there are more than 100 values. Important: The colors in Customize apply to the current visualization only.
Additional mapping	If the Type is By dimension, you can use this option to define conditions to apply colors. To do so, you must define a condition. For example, you can define to associate a color when the value is inferior (<) to the expected value. matches: allows you to specify a regular expression to choose the color or the icon to apply, depending on the value. Important: What you define in Additional mapping has precedence over what you define in Mapping.
Splitting strategy	If the Type is By measure, you can use this option to split the color scale into intervals. There are 2 splitting strategies: Gradient: Split the color scale into N gradient colors. Discrete: Split the color scale into N discrete intermediate colors.

3. **Note:** There are priorities if you define several colors for the same field value. Manual customizations have priority over display rules.

Drill-Down and Drill-Up on Dimensions

You can configure your chart with 2 Axis dimensions to let users drill-down and drill-up from one dimension to another.

1. In the **Properties** panel, go to [?](#) **Data**.
2. In the **Axis** section, click [?](#) to add a **Dimension**.
3. For **Field**, select the field to use as X axis.
4. **Optional:** For **Label**, enter the label to display instead of the field name.

Note: To localize this text in other languages, click [?](#), and select a translation key. You define keys and values in the **Localization** panel.

Important: The visualization shows in order of priority: the **X-axis Title**, the dimension **Field Label** (defined with this option), and the **Field**.

5. Repeat steps 2 and 3 to define another **Axis**.

The upper right of the chart shows up and down arrows [?](#) [?](#) to let users switch from one dimension to another.

6. **Optional:** To add a chart filtering to the drill-down, add an **Effect on chart Click**.

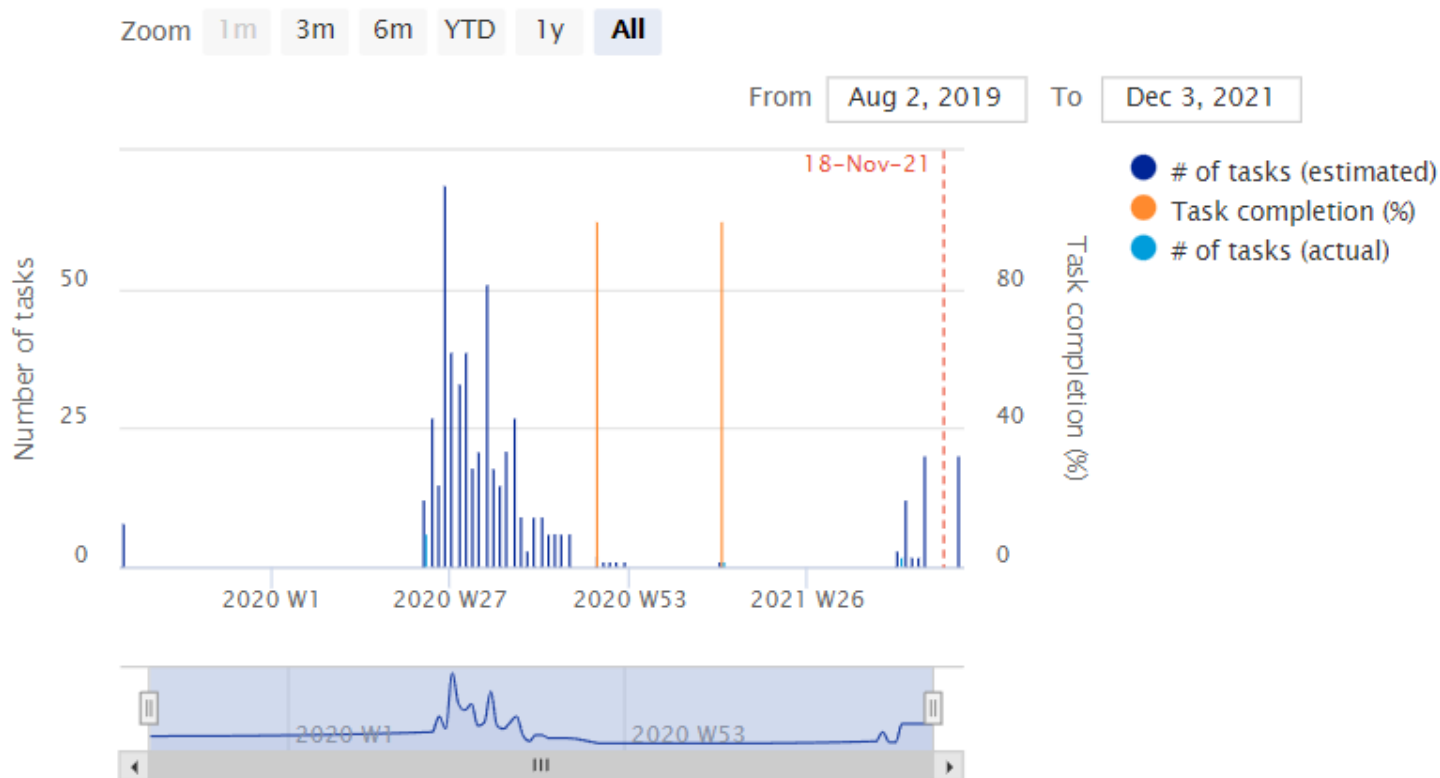
Add Time Series to the Chart

Basic charts can display time series, which offer navigation capabilities allowing you to filter or zoom on time. Before you begin: The first **Axis** dimension must be of type **Date**, **Datetime**, or **LocalDatetime**.

- 1. In the **Properties** panel, go to  **Data**.
- 2. In the **Time Series Options** section, configure the following properties.

Option	Description
Use time series	Activates the display of time series
Show navigation	Add a range selector and a navigator to the time series.
Current date indicator	Adds a line to indicate the current date on the X-Axis.
Select time range	Select a predefined time range. This time range applies to data before the current date. For ex
Current date indicator options	Edit the current date indicator display.

Leaf Task completion by estimated and actual finish dates



Note: The time series cannot show a time range below 5 points (for example, you cannot see a range of 3 points).

Define Display Options for the Axes

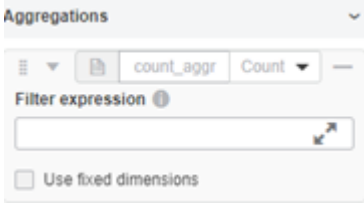
You can define several display options to configure the X-axis and Y-axis of your chart.

1. In the **Properties** panel, go to **Data**.
2. Go to the **Axis options** section.
3. Configure the **X-axis** display with the following options.

Option	Description
Title	Static value: Enter the label to show next to the value. Translatable value: To localize the title in other languages, you can select a translation key. You define keys and translations in the Translations panel. Expression value: To configure a dynamic title, you can enter an expression based on the expression language. In your expression, you can use a Variable with <code>VARIABLES.<variable name></code> and a Translation key with <code>TRANSLATION.<translation key></code>. For example, you can make a dynamic title like <code>Number of cars sold in {{country}}</code>, where the title string is <code>Number of cars sold in</code> and the variable is <code>country</code>. If the variable value is <code>France</code>, you get <code>Number of cars sold in France</code> if English is the preferred language, <code>Nombre de voitures vendues en France</code> if French is the preferred language, and so on. You can also use Aggregations coming from the RESULT SET of your data query.

	Example 1: You can configure a count in the title to get <code>Number of cars: <count value></code>. In the RESULT SET,  In the visualization, use the <code>count_aggr</code> aggregation in the ? Expression value: <code>"Number of cars: " + cars.co</code> Example 2: You can configure the title to display <code>Car sales in: <set of country values></code>. In the RESULT S In the visualization, use the <code>countries_aggr</code> aggregation in the ? Expression value: <code>"Car sales in: " + cars</code>
Position	Select where to show the X-axis title. It can be at the Bottom (default postion) or at the Top of the axis.
Padding	For Bar and Column charts, you can specify the space to apply between each bar of the chart. The space between b

4. Configure the **Y-axis** display with the following options.

Option	Description
Title	? Static value: Enter the label to show next to the value. ? Translatable value: To localize the title in other languages, you can select a translation key. You define keys and ? Expression value: To configure a dynamic title, you can enter an expression based on the expression language. In your expression, you can use a ? Variable with <code>VARIABLES.<variable name></code> and a ? Translation key with For example, you can make a dynamic title like <code>Number of cars sold in {{country}}</code>, where the title string If the variable value is <code>France</code>, you get <code>Number of cars sold in France</code> if English is the preferred language, You can also use Aggregations coming from the RESULT SET of your data query. Example 1: You can configure a count in the title to get <code>Number of cars: <count value></code>. In the RESULT SET  In the visualization, use the <code>count_aggr</code> aggregation in the ? Expression value: <code>"Number of cars: " + cars.</code> Example 2: You can configure the title to display <code>Car sales in: <set of country values></code>. In the RESULT In the visualization, use the <code>countries_aggr</code> aggregation in the ? Expression value: <code>"Car sales in: " + car</code>

Position	Select where to show the X-axis title. It can be at the Bottom (default position) or at the Top of the axis.
Scale	Select how to display the axis, that is, either with a linear or a logarithmic scale.
Display as	Select whether to display values or percentages on the axis.
Min	Specify the minimum value to display on the Y-axis. By default, Data Perspective Studio generates the display automatically.
Max	Specify the maximum value to display on the Y-axis. By default, Data Perspective Studio generates the display automatically.

Split a Chart with Multiple Measures

You can split a chart with multiple measures into several connected charts within a grid layout made of several cells.

1. In the **Properties** panel, go to **Data**.
2. Go to the **Grid** section, and configure how to split the chart in **Cells**.

Option	Description
Display on	If you have several axes defined in Y-axis , you can choose for which axis you want to display the chart in the cell.
Column index	Select the column number in the grid. This is a 0-based index.
Row index	Select the row number in the grid. This is a 0-based index.

For example, you can split a column chart with 3 measures into 3 charts, displayed on 2 rows and 2 columns.



Split a Chart by Dimension Categories

You can split a chart with another dimension or field, and display each dimension or field category within a grid.

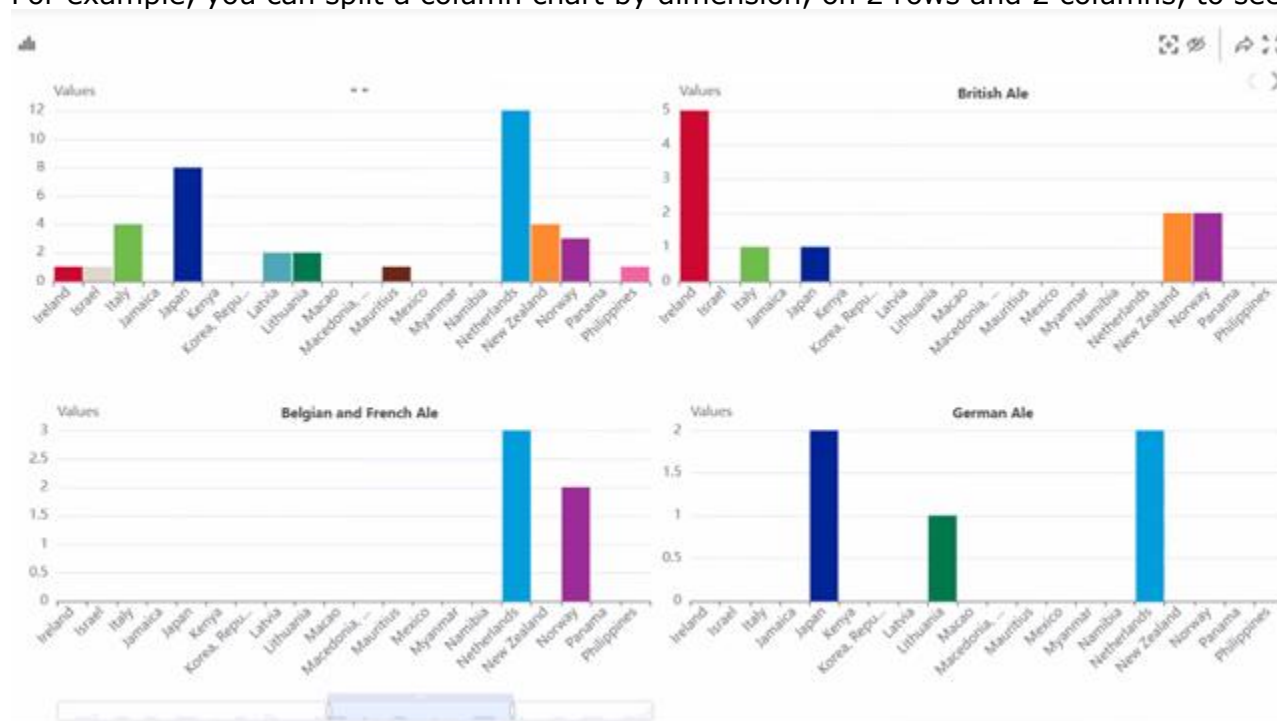
1. In the **Properties** panel, go to [?](#) **Data**.
2. Go to the **Grid** section, and select **Split by dimension**.
3. From **Field or dimension** section, select the dimension on which you want to split the chart.
4. **Optional:** Click **More options** to open the advanced configuration properties.

Option	Description
Limit	Select the maximum number of categories to display for the dimension. When the number of categories crosses this limit, the chart creates a pagination gathering all the remaining categories.
Sort by	Select how to sort dimension categories. Alphanum: sorts the categories alphanumerically. Custom: lets you specify how to sort the categories contained in the dimension by defining Ordered values. Measure: lets you select any aggregation defined either in ? Data > Values > Measures or in the RESULT SET operator. Click ? or ? to sort in ascending or descending order.
Format	You can select a text format: No formatting, lower case, UPPER CASE, Capital on first letter + no change (capitalizes the first letter of each word), Capital On All First Letters (capitalizes the first letter of each word).

5. From **Cells**, configure how to split the chart in the grid.

Option	Description
Display on	If you have several axes defined in Y-axis , you can choose for which axis you want to display the chart.
Max number of columns	Select the max number of columns to display in the grid. You can add up to 16 columns.
Max number of rows	Select the max number of rows to display in the grid. You can add up to 8 rows.

For example, you can split a column chart by dimension, on 2 rows and 2 columns, to see the values of vari



Note: You can use the slider at the bottom of the chart to browse through its X-axis dimension. You can also

Show Underlying Data in a Data Panel

You can add a **Show underlying data** action in the **?** **Actions** menu of most charts. On-click, Data Viewers o

1. In the **Properties** panel, go to **?** **Data**.
2. In the **Data Panel** section, select **Enable Data Panel**.
3. From **Fields**, add the fields that you want to see as table columns in the data panel.
4. **Optional:** Select **Allow to reorder columns** if you want Data Viewers to be able to reorder columns b

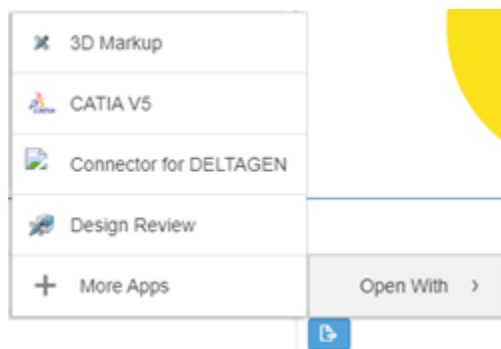
5. **Optional:** From **Buttons**, define the icons (and their on-click effects) to show in an **Actions** column.
6. **Optional:** Select **Add drag button in the first column** to add a drag icon  in that column for users to

- a. In **Object ID source**, select the field to use as data source.
- b. In **Object type source**, specify the name of the object to drag. For example, `VPMReference`.
- c. **Optional:** In **Object path source**, specify the field which contains the definition of the `path`

Note: The destination widget uses the `path` attribute to get the context of the dragged object. The field must be a tuple table, where each tuple defines a `resourceid` attribute, and a `type`

```
[{
  "resourceid": "7ED583CD0000168C63C6F2E200020D1F",
  "type": "VPMReference"
},
{
  "resourceid": "7ED583CD0000168C63C6F2E300043C51",
  "type": "VPMInstance"
},
{
  "resourceid": "7ED583CD0000168C63C6F2E200020D56",
  "type": "VPMReference"
}]
```

- d. For **3DEXPERIENCE Service**, select the variable, which contains the id of the **3DEXPERIENCE**
7. **Optional:** Select **Add "Open with" in the actions column** to add a button allowing Data Viewers to open
 - a. **Optional:** Enter the **Label** to show on the **Open with** button.
 - b. In **Icon**, specify the button icon.
 - c. **Optional:** You can also add a tooltip and a theme.



8. **Optional:** From **Effects (On checkbox click)**, you can define an interaction with a 3D Widget outside

Note: To do the opposite, that is, click an object in a 3D Widget and refresh the Data Perspective view

9. **Optional:** From **Effects (On 3D Selection)**, you can define how the data panel reacts, when the Data

Underlying data - Opened issues			
Objects		Values	
Name	Priority	Maturity state	Actions
ISS-0000001	Pre-assigned	Draft	
autodata-0330_ProjectIssue-1	High	In Approval	
autodata-0330_ProjectIssue-2	High	In Approval	
autodata-0330_ProjectIssue-3	High	In Approval	
autodata-0330_ProjectIssue-4	High	In Approval	
autodata-0330_ProjectIssue-5	High	In Approval	

Reduce the Scope of Dimensions with Local Filters

For a visualization that displays data for a dimension (for example, in a chart axis), it may be useful to focus

For example, if you have a **priority** dimension that contains **Very Minor**, **Minor**, **Major**, and **Critical** priorities,

Important: Unlike **Filters**, you cannot select **Local Filters** on the visualization UI.

1. In the **Properties** panel, go to **Data**.
2. In the **Local Filters** section, specify the **Dimensions** for which you want to filter data out as follows:
 - a. From **Field**, select the dimension on which you want to apply the filtering.
 - b. From **Filtering mode**, select **Include** or **Exclude**.
 - c. From **Values**, select the set of values to include or exclude from the dimension.

For example, for the **Status** dimension, you can **exclude NULL** values.

LOCAL FILTERS

Dimensions

▼ Status

Field

Status ▼

Filtering mode

Exclude ▼

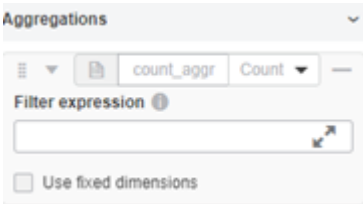
Values

NULL Select values ▼

Configure the Display

You can configure the display using several options.

- 1. In the **Properties** panel, go to [?](#) **Style**.
- 2. In **Title**, specify the visualization title.

Option	Description
? Static value	To enter a static textual value.
? Translatable value	To localize the title or label in other languages, you can select a translation key. You define keys and tr
? Expression value	To configure a dynamic title or label, you can enter an expression based on the expression language. In your expression, you can use a ? Variable with <code>VARIABLES.<variable name></code> and a ? Translation For example, you can make a dynamic title like <code>Number of cars sold in {{country}}</code>, where the If the variable value is <code>France</code>, you get <code>Number of cars sold in France</code> if English is the preferred You can also use Aggregations coming from the RESULT SET of your data query. Example 1: You can configure a count in the title to get <code>Number of cars: <count value></code>. In the R  In the visualization, use the <code>count_aggr</code> aggregation in the ? Expression value: <code>"Number of cars: "</code> Example 2: You can configure the title to display <code>Car sales in: <set of country values></code>. In th In the visualization, use the <code>countries_aggr</code> aggregation in the ? Expression value: <code>"Car sales in:</code>

- 3. **Optional:** In **Icon**, select the icon that represents the visualization.
- 4. **Optional:** Select the **Theme** to apply on this visualization.

By default, Data Perspective Studio applies the theme of the parent page to the visualization.

You can define your own themes in [?](#) **Resources** > [?](#) **Style**.
- 5. **Optional:** Select **Hide header**, if you want to show the visualization without header (this hides the **T**

6. **Optional:** Select **Hide breadcrumb**, if you want to hide breadcrumb navigation in the visualization.
7. **Optional:** Select **Minimizable** to add a **Minimize** option in the **Actions** menu, allowing users to collapse the visualization. You can also select **Minimized by default** if you want the default display to show a collapsed view of the visualization.
8. **Optional:** Select **Customize font size** to change the default font size (14 pixels) for all chart labels, in the visualization.
9. **Optional:** From **Tooltip trigger type**, select what you want to show for hovered information in chart tooltip.
 - **Axis** (default option): When you hover above the X-axis, all point values display in a single tooltip.
 - **Item**: When you hover above the X-axis, each point value displays in a separate tooltip.

Configure the Chart Legend

Several options let you configure the display of the chart legend. You can choose to show or hide the legend to the end users.

Important: When you click a legend entry to apply a filter, Data Perspective Studio hides this entry from the chart but does not hide the legend itself.

1. In the **Properties** panel, go to **Style**.
2. **Optional:** In the **Legend** section, select **Hide legend** to hide the chart legend to the end users. This also hides the **Show Legend** option in the **Actions** menu for end users.
3. For **Position**, select how to show the legend on the chart.

If you select **Top**, **Bottom**, **Left**, or **Right**, the **Responsive legend position** overrides the legend position.

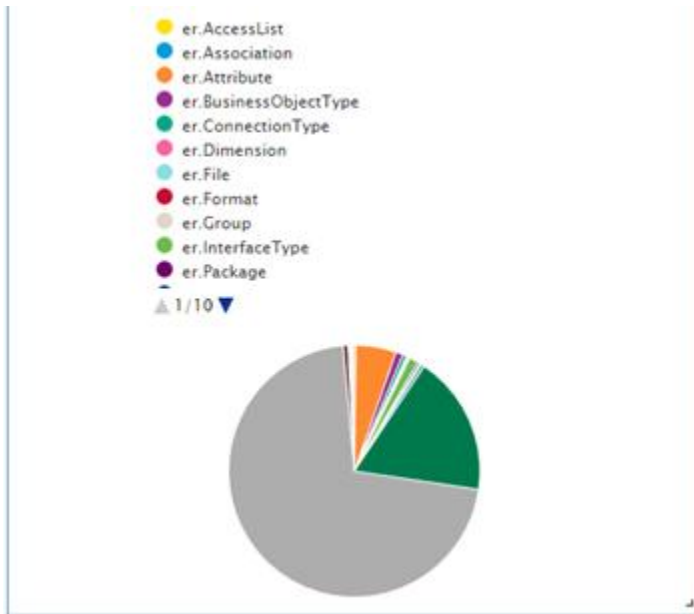
If you select **Floating**, you can define a floating position with the following options:

Option	Description
Horizontal alignment and Vertical alignment	Align the legend horizontally and vertically.
Horizontal offset	Specifies the legend offset relative to its horizontal alignment. Negative values move the legend to the left.
Vertical offset	Specifies the legend offset relative to its vertical alignment. Negative values move the legend upwards.
Layout	Choose a vertical (in column) or horizontal (in a row) layout for the legend.

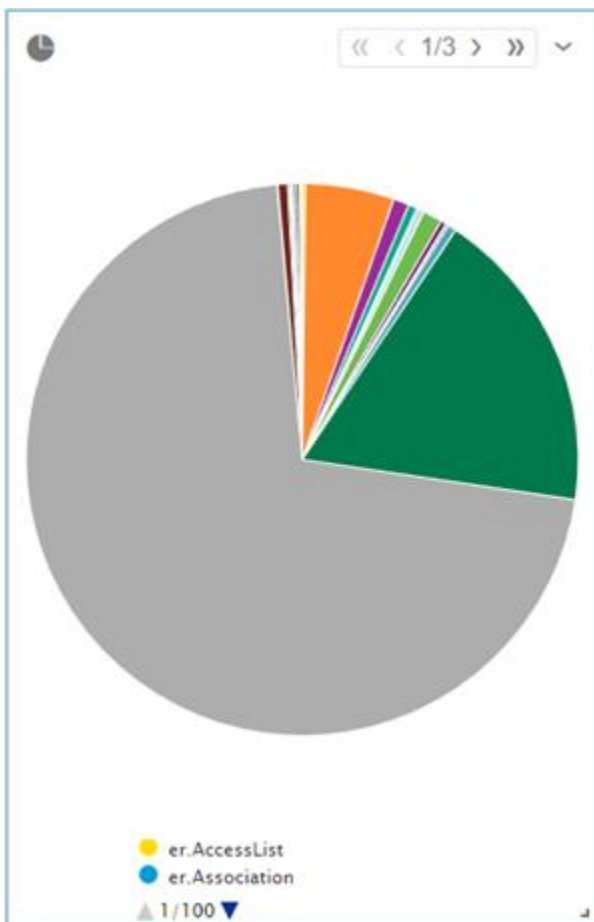
4. For **On legend click**, define the legend and chart behavior when you click a legend entry.
 - If you select **Show series**, clicking a legend entry shows the corresponding chart series only (y-axis).
 - If you select **Hide series**, clicking a legend entry hides this series from the chart. The legend displays the series name.

This example shows a pie chart for which the selected legend **Position** is **Top**.

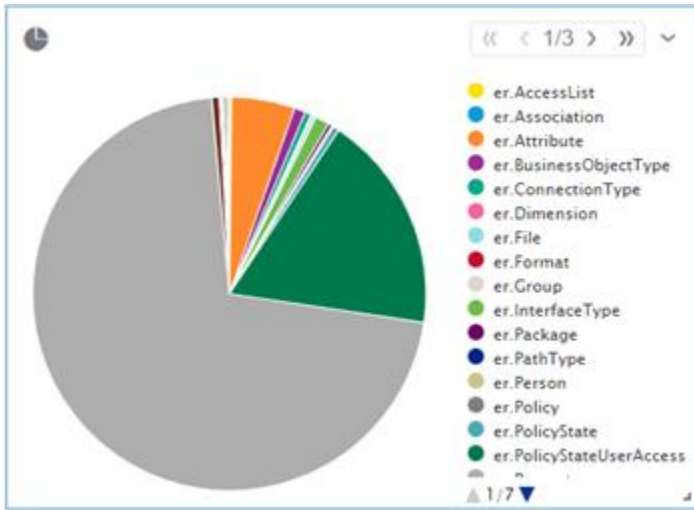
By default the legend appears at the top.



If the visualization width is below 400px and the widget height is above 400px, the legend shows at the bot



If the visualization height is below 400px, the legend shows at the right of the chart. The responsive design



Add Effects On Chart Click

You can add effects when users click visualization elements.

1. In the **Properties** panel, go to **Actions**.
2. Click **+** to add a new effect.
3. Select the effect **Behavior**.

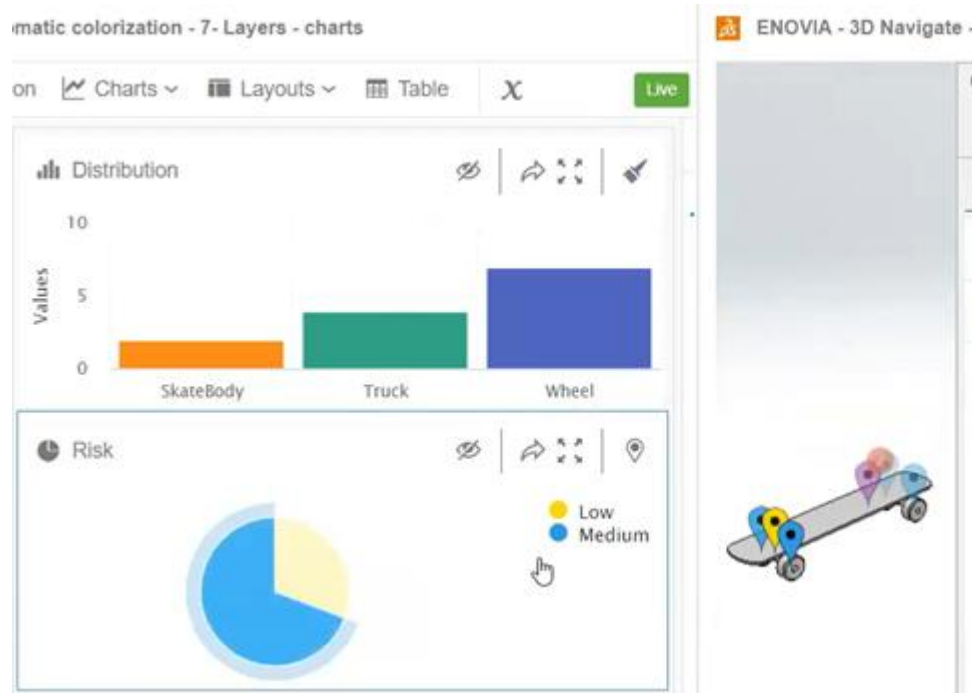
Option	Description
Filter	Filters page data with the content dropped on the page. Information source: Specify the field to filter data. Filter scope: Select a filter scope defined in Settings > Filter Scopes. By default, the general scope is selected.
Set variable	Specify the variable that stores the data. Target studio variable: Select the data query variable where Data Perspective Studio stores source data.
Toggle variable	With this option, when you click a selected item, it clears it.
Navigate to	You can choose between the following modes: Page (default mode): Select the destination page to open when you click an object. The page is opened in the same browser tab. Static URL: Enter the link to the target page. Links open a new browser tab and there is no back button. String variable: Select a string variable containing the URL to open. You define the URL in the String variable field.
Color washing	Applies color washing on a related Target viewer, which can be a 3D viewer or a City viewer when you click an object. 3D viewer (ENOVIA widgets such as 3D Navigate, or 3D Compose) configuration:

Important: This effect requires a link between your Data Perspective and the 3D widget. From the Perspective Studio widget header, click [?](#) actions, and select [?](#) **Link with Widgets**. Then in the 3D widget, click [?](#) **Link** button and click **Apply**.

- For **Information type**, select:
 - **Reference ID:** It is useful to identify one or several occurrences of a part in the 3D model.
 - **Occurrence Path:** It is useful when your 3D model uses the same part several times. Select the **Occurrence Path Source**.
- **Optional:** The **Enable cumulative mode** option (selected by default) creates a dedicated layer for each data point.

Note: If the 3D widget already contains layers, the cumulative mode merges the new layers with the existing ones.

Color washing with cumulative mode (example with **Pie chart**)



- **Optional:** The **Enable transparency** option enables transparency for all the 3D parts in the 3D widget.

City viewer (for an Urban Environment app like City Play or City Discover) configuration:

- For **Identifier source**, select an attribute containing the identifiers of the items to colorize.
- For **Dataset**, select the dataset to colorize in the city viewers.

Tip: You can define a [?](#) variable value if you want to use a variable as a constant value. This is useful if you have a lot of visualizations and do not want to manually define a value for each of them.

- **Optional:** Select **Enable transparency** to enable transparency for all the items in a data layer.

Pinpoint in 3D

You can display Points of Interest (POI) in widgets having 3D capabilities (ENOVIA widgets such as 3D Explorer).

Important: This effect requires a link between your Data Perspective and the 3D widget. From the Perspective Studio widget header, click  actions, and select  **Link with Widgets**. Then in the 3D widget header, click the  **Link** button and click **Apply**.

- For **Information type**, select:
 - **Reference ID**: To identify one or several occurrences of a part in the loaded model.
Note: To use this option, also select the **Reference ID Source**.
 - **Occurrence Path**: It is useful when your 3D model uses the same part several times.

Note: To use this option, also select the **Reference ID Source**.

Note: To use this option, also select the **Occurrence Path Source**.

- Optionally, select a field containing the **Local position coordinates** of the POI. By default

In some cases, multiple layers of POIs are projected at once. If there are multiple POI a Semantic Graph Index field. The values in this field must be arrays of 16 floats: the

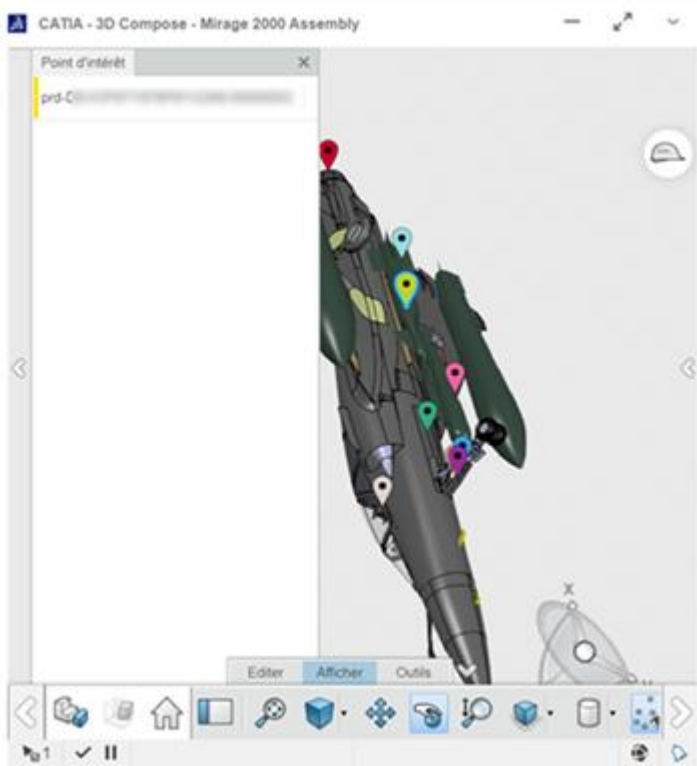
- The **Enable cumulative mode** option (selected by default) creates a dedicated layer in the

Note: If the 3D widget already contains layers, the cumulative mode merges the new

- For **Points of Interest**, indicate the **Identifier source**, that is the field to use as URI/physical location.
- Optionally, add **Graphic layers**. A layer represents a group of pins.

You can specify the **Name**, the **Color**, and the **Icon** (by default a pinpoint icon) to asso

Note: When similar Points of Interest are very close to one another on the product structure, they may be combined into a single Point of Interest.

	
Send message to other apps	This effect allows the Data Perspective to send messages to a target web app. Important: To receive the Data Perspective messages, the target web app must use the Publish and Subscribe Protocol to subscribe to events. You can also implement the WidgetLink feature. In the visualization, configure a Send message to other apps on-click effect: • For Topic, specify the topic that allows the Data Perspective and the target web app to communicate. • For Payload, define the attributes that you want to send to the target web app. Note: If you have implemented the WidgetLink feature instead of using the Publish and Subscribe protocol, you must use the WidgetLink feature.

Add Effects (On Apply selection click) for Multiple Selection

You can add effects when users click **Enable multi selection** in the top bar of the chart to select a range of series.

Before you begin: In [? Data](#), you must have at least one **Measure** in your chart.

1. In the **Properties** panel, go to [? Actions](#).
2. Under **Effects (On Apply selection click)**, click [?](#) to add a new effect.
3. Select the effect **Behavior**.

Option	Description
Filter	Filters page data with the content dropped on the page. Information source: Specify the field to filter data. Filter scope: Select a filter scope defined in Settings > Filter Scopes. By default, the general scope is selected.
Navigate to	You can choose between the following modes:  Page (default mode): Select the destination page to open when you click an object.  Static URL: Enter the link to the target page. Links open a new browser tab and then the target page.  String variable: Select a string variable containing the URL to open. You define the string variable in the Variables section.
Send message to other apps	This effect allows the Data Perspective to send messages to a target web app. Important: To receive the Data Perspective messages, the target web app must use the Publish and Subscribe Protocol to subscribe to events. You can also implement the WidgetLink feature. In the visualization, configure a Send message to other apps on-click effect: - For Topic, specify the topic that allows the Data Perspective and the target web app to communicate. - For Payload, define the attributes that you want to send to the target web app. Note: If you have implemented the WidgetLink feature instead of using the Publish and Subscribe Protocol, you can use the Send message to other apps effect to send messages to the target web app.

In the top bar of your chart, you can now click  **Enable multi selection**.

The cursor becomes a  cross and you can select a range of series in your charts (you can slide the selection).

When you click  **Apply**, it applies the interaction effect defined for the visualization.

Note: To clear the selection, click .

Configure the Actions Menu

You can create groups of menu actions to show either directly in the visualization header, or in the  **Actions** menu.

In the visualization, you have default menu groups with:

- Generic actions:  **Full screen**,  **Maximize / Minimize**,  **Export visualization**
- Actions which are specific to the selected visualization.

You can also create your own menu groups. For all menu groups, you can:

- Click  to add custom actions, reorder actions, remove actions.
- Define the display, the position, and the priority of the menu group compared to other menu groups.

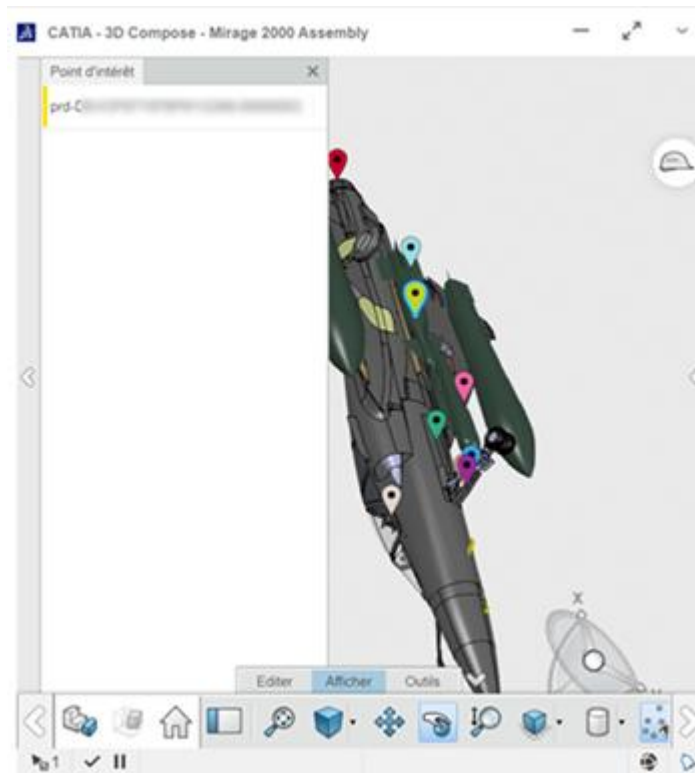
1. In the **Properties** panel, go to  **Actions**.
2. The **Visualization menu** section has predefined **Menu groups**, which contain default actions. You can

Action	Description
 Full screen	Shows the visualization in full screen in your widget.
 Maximize / Minimize	Allows you to collapse or expand the visualization view in the Data Perspective. This action is available when you select the  Style tab > Minimizable option.
 Export visualization	Exports a snapshot of the visualization view in png.
 See underlying data	Allows you to open data in a data panel to get more details. This action is available when you select the  Data tab > Enable data panel option.
 Show / hide legend	Shows or hides the chart legend. This action is available only if the  Style tab > Hide legend option is not selected.
 Go to dimension	Switches the axis dimensions. This action is available when you define 2 Axis dimensions in the  Data tab, to drill down or drill

3. **Optional:** To add a custom action to the menu group, click  next to **Menu actions**.
 - a. For **Icon**, specify the icon to associate with the action.
 - b. For **Label**, enter the action label.
 - c. In **Effect (On click)**, select the effect to run when you click an object.

Option	Description
Navigate to	You can choose between the following modes: d.  Page (default mode): Select the destination page to open when you click an object. e.  Static URL: Enter the link to the target page. Links open a new browser tab and the

	f. String variable: Select a string variable containing the URL to open. You define the
Open/Close advanced filters panel	Opens or close an advanced filters panel outside of the Data Perspective. When you open this panel, it replaces the Tags panel on the left of the 3DEXPERIENCE platform.
Clear variable	Clears the variable that stores the data. Target studio variable: Select the data query variable where Data Perspective Studio stores source data.
Reset variable	Resets the variable that stores the data to its Initial value . Target studio variable: Select the data query variable where Data Perspective Studio stores source data.
Pinpoint in 3D	You can display Points of Interest (POI) in widgets having 3D capabilities (ENOVIA widgets such as 3D Modeler, 3D Explorer, 3D Viewer, etc.). Important: This effect requires a link between your Data Perspective and the 3D widget. From the Data Perspective Studio widget header, click actions, and select Link with Widgets. Then in the Link with Widgets dialog box, click the Link button and click Apply. g. For Information type, select: • Reference ID: To identify one or several occurrences of a part in the load tree. Note: To use this option, also select the Reference ID Source. • Occurrence Path: It is useful when your 3D model uses the same part several times. Note: To use this option, also select the Occurrence Path Source. h. Optionally, select a field containing the Local position coordinates of the POI. By default, the Local position coordinates are selected. In some cases, multiple layers of POIs are projected at once. If there are multiple layers, select a Semantic Graph Index field. The values in this field must be arrays of 16 floats. i. The Enable cumulative mode option (selected by default) creates a dedicated layer for each POI. Note: If the 3D widget already contains layers, the cumulative mode merges the layers. j. For Points of Interest, indicate the Identifier source, that is the field to use as URI/parts name. k. Optionally, add Graphic layers. A layer represents a group of pins. You can specify the Name, the Color, and the Icon (by default a pinpoint icon) to each layer. Note: When similar Points of Interest are very close to one another on the product, they are merged.

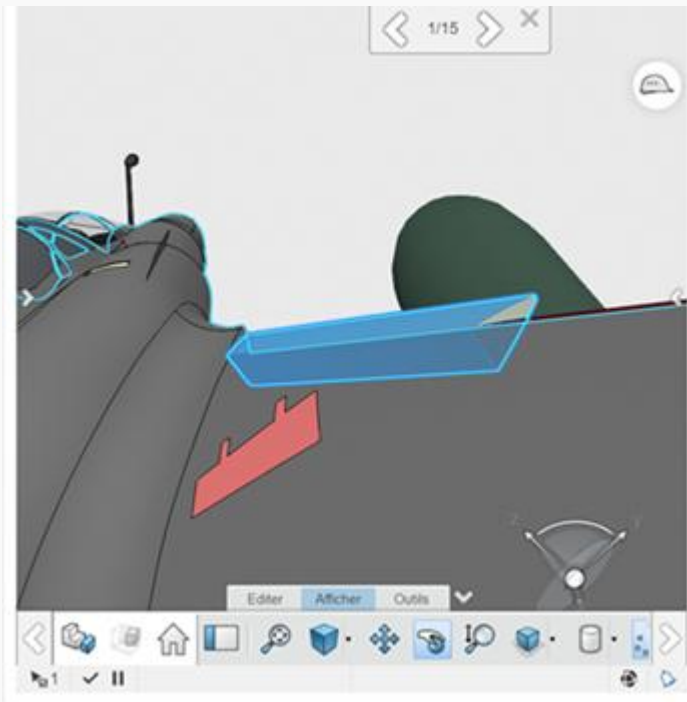


Highlight in other apps

You can add this effect to highlight elements in target **3D viewer** or **City viewer** apps.

3D viewer: To highlight a visualization element on a part of a product structure displayed in EN

- l. **Optional:** Select **Focus on 3D Part**, if you want to focus on the part in the product
- m. For **Information type**, select:
 - **Reference ID:** It is useful to identify one or several occurrences of a part
 - **Occurrence Path:** It is useful when your 3D model uses the same part several times. Select the **Occurrence Path Source**.



City viewer: To highlight a visualization element in a city viewer application, like City Play or

- n. **Optional:** select **Focus on item**, if you want the camera to focus on the selected item.
- o. For **Identifier source**, select an attribute containing the identifiers of the items to highlight.
- p. For **Dataset**, select the dataset to highlight in the city viewers.

Tip: You can define a [?](#) variable value if you want to use a variable as a constant value. This is useful if you have a lot of visualizations and do not want to manually set a value for each of them.

Color washing

Applies color washing on a related **Target viewer**, which can be a **3D viewer** or a **City viewer** when

3D viewer (ENOVIA widgets such as 3D Navigate, or 3D Compose) configuration:

Important: This effect requires a link between your Data Perspective and the 3D widget. From the Perspective Studio widget header, click [?](#) actions, and select [?](#) **Link with Widgets**. Then in the widget configuration, click [?](#) **Link** button and click **Apply**.

- q. For **Information type**, select:
 - **Reference ID:** It is useful to identify one or several occurrences of a part.
 - **Occurrence Path:** It is useful when your 3D model uses the same part several times. Select the **Occurrence Path Source**.
- r. **Optional:** The **Enable cumulative mode** option (selected by default) creates a dedicated layer for the color washing.

Note: If the 3D widget already contains layers, the cumulative mode merges the new layer with the existing ones.

Color washing with cumulative mode (example with **Pie chart**)

Automatic colorization - 7 - Layers - charts

on Charts Layouts Table X Live

Distribution

10

5

0

Values

SkateBody

Truck

Wheel

Risk

Low

Medium

ENOVIA - 3D Nav



s. Optional: The **Enable transparency** option enables transparency for all the 3D pa

City viewer (for an Urban Environment app like City Play or City Discover) configuration:

t. For **Identifier source**, select an attribute containing the identifiers of the items to c

u. For **Dataset**, select the dataset to colorize in the city viewers.

Tip: You can define a ? variable value if you want to use a variable as a const

value. This is useful if you have a lot of visualizations and do not want to manu

value for each of them.

v. Optional: Select **Enable transparency** to enable transparency for all the items in a

4. From **Group display**, define the display in the header by selecting an option.

Option	Description
Menu	Adds the menu group in an ? Actions menu.
Responsive	Adds the menu group in the visualization header. If the visualization is not wide enough to show the menu group
Visible	The menu group is always visible in the visualization header.

- From **Group position**, define the position (**Left**, **Center**, **Right**) of the menu group in the visualization header.
- Optional:** Define a **Group priority**, if the **Group display** is **Responsive**. The group with the highest priority is displayed first.

If there is not enough space to show all groups in the visualization header, the visualization shows groups in a scrollable menu.

Add Target Lines to the Chart

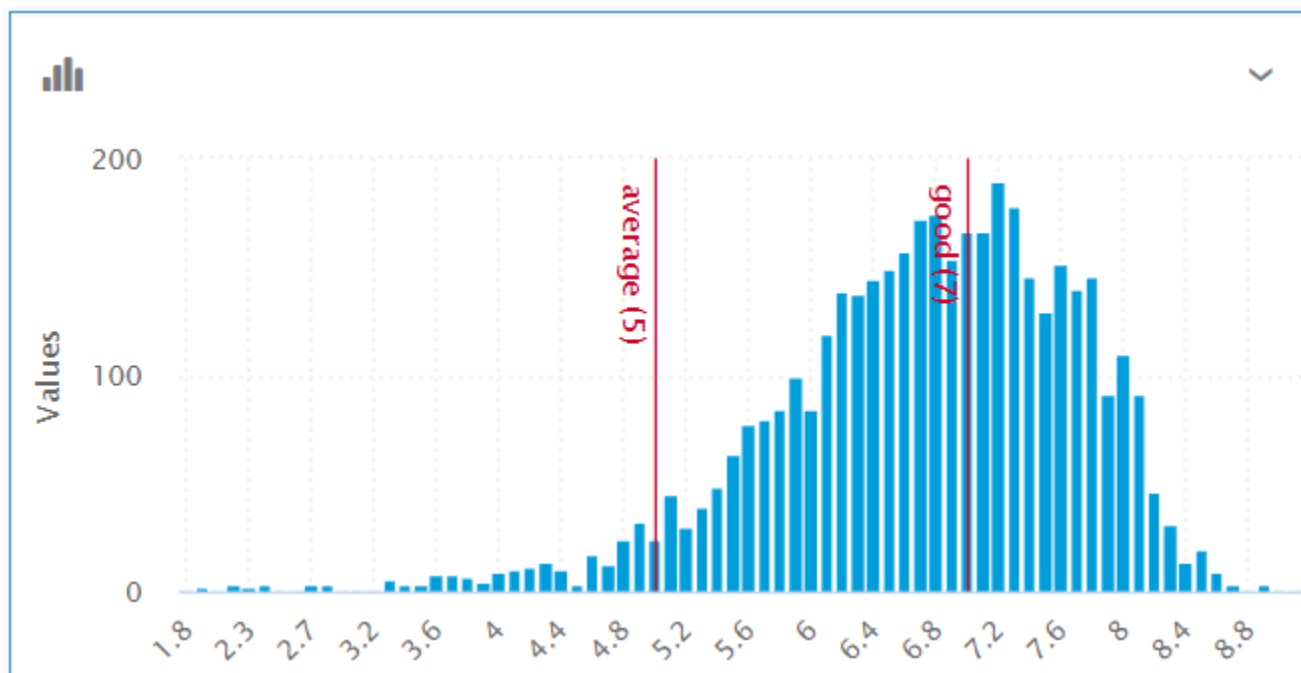
You can add one or several target lines to the chart. The target line is perpendicular to the axis on which it is displayed.

- In the **Properties** panel, go to  **Layers**.
- In the **Lines** section, click  to add a new target line.
- Configure the target line with the following properties.

Property	Description
Type	Select how to define the target line: Static: To specify the value manually in the Value field displayed below. Dynamic: To retrieve the value dynamically from a Result set field. Select this field from Value. Variable: To retrieve the value dynamically from the Variable specified below. For example, you can use <code>sum(sales)</code>.
Direction	Choose to show an Horizontal or a Vertical line. For the Vertical direction: - You must sort data alphanumerically. - You must use numeric or date dimensions only. - The direction uses a continuous axis instead of a discrete one.
Label	You can specify the line label: - Manually for a Static Line type. Note: To localize this label in other languages, click , and select a translation key. You define keys in the Translations section. - Dynamically from a field if you selected a Dynamic Line type.
Result set	If the Type is Dynamic , select the result set that contains the aggregation you want to use.
Aggregation	If the Type is Dynamic , select the value for which you want to draw a line.
Display on	Select on which axis to add the target line.

Variable	If the Type is Variable , select the variable that gives its value to the target line.
Color	Select the line color from the color picker.
Line style	Select whether to display a solid, a dot, or a dashed line.
Text display	Select the text to display for the line. • Name shows the value defined in the Label property. • Value shows the value defined in the Value property. • Name and value show the values defined in the Label and Value properties. • No text hides both the values defined in the Label and Value properties.
Text position	Select where to display the text: above or below the line, on the left, or on the right.

In the following screenshot, you can see two static target lines in a column chart, to highlight thresholds.



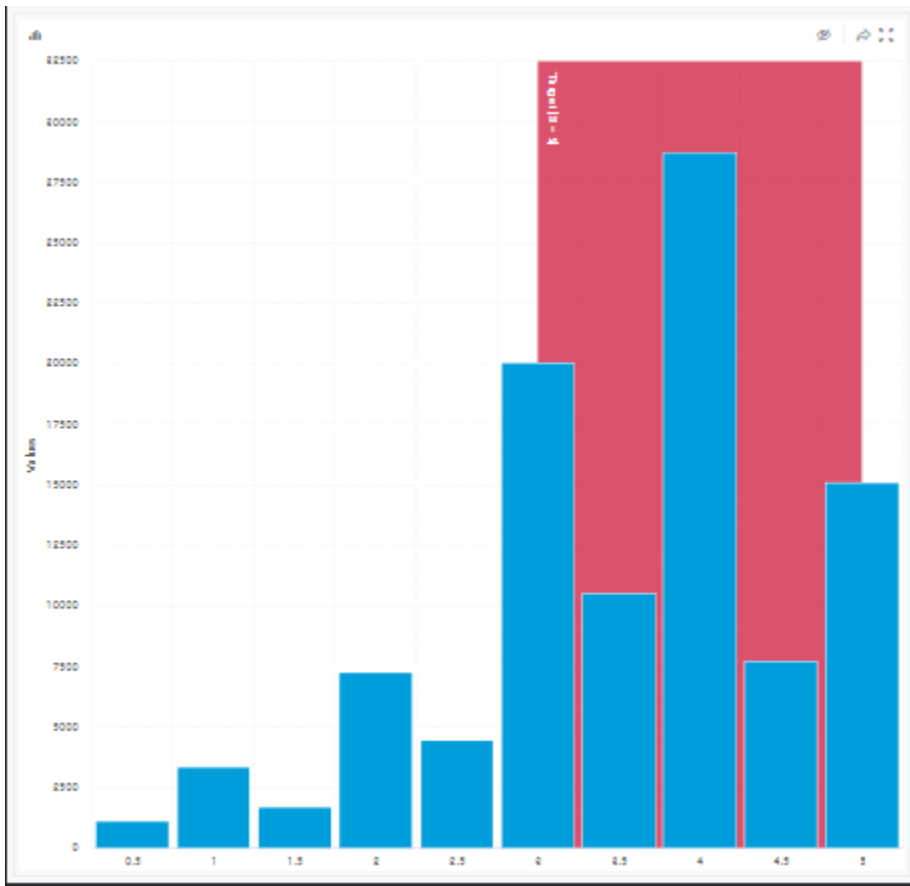
Add Areas to Chart Targets

You can define colorized areas on chart targets to highlight a specific range of values on an axis. These areas

1. In the **Properties** panel, go to **Layers**.
2. In the **Areas** section, click **+** to add a new area.
3. Configure the target area with the following properties.

Option	Description
Type	Select how to define the target area: • Static: To specify the value manually in the Min and Max fields displayed below. • Dynamic: To retrieve the value dynamically from a Result Set, with Min Aggregation and Max Aggregation.
Direction	Choose to show an Horizontal or a Vertical line. For the Vertical direction, the axis must use a numerical value or a date, and data must be sorted alphanumerical.
Label	You can specify the area label: • Manually for a Static Line type. Note: To localize this label in other languages, click , and select a translation key. You define keys in the Translations section. • Dynamically from a field if you selected the Dynamic type.
Result set	If you selected the Dynamic type, select the Result Set which contains the Aggregation you want to use.
Display on	Select on which axis to add the target area.
Color	Select the area color from the color picker.
Text display	Select the text to display for the area. • Name shows the value defined in the Label property. • Value shows the value defined in the Value property. • Name and value show the values defined in the Label and Value properties. • No text hides both the values defined in the Label and Value properties.
Text position	Select where to display the text in the area (Top - Left , Top - Right , Bottom - Left , Bottom - Right).

In the following screenshot, you can see a red area in the background of a column chart. It highlights the range of values between the minimum and maximum values defined in the target area.



Add Marker Points to the Chart

You can define marker points for specific values or events on your chart. These points can appear on the default chart. Marker points are typically useful to indicate important events or values on which you want to draw your user's attention.

1. In the **Properties** panel, go to **Layers**.
2. In the **Marker points** section, click **+** to add a new marker point.
3. Configure the marker point with the following properties.

Property	Description
Type	Select how to define the marker point: • Static: To specify the value manually in the X-axis value and Y-axis value fields displayed below. • Dynamic: To retrieve the value dynamically from Result set fields. Select these fields in the X-axis and Y-axis dropdowns. • Variable: To retrieve the marker point values dynamically from the X-axis variable and Y-axis variable dropdowns.
Display on measure	Select on which measure to add the marker point.

	Warning: You cannot use a measure that is: - Stacked with another measure, using the Stack with measure option. - Split into several series, using the Breakdown option.
Display on dimension	Select on which dimension to add the marker point. The dimension must contain data of numeric or d
Label	You can specify the marker point label: - Manually for a Static marker point. Note: To localize this label in other languages, click , and select a translation key. You de - Dynamically from a field if you selected a Dynamic marker point.
X-axis value	If the Type is static, enter the X-axis value. If the Type is Dynamic, select the field containing the X-axis value.
Y-axis value	If the Type is static, enter the Y-axis value. If the Type is Dynamic, select the field containing the Y-axis value.
Result set	If the Type is Dynamic , select the result set that contains the fields you want to use.
X-axis variable	If the Type is Variable , select the variable that gives the X-axis value to the marker point.
Y-axis variable	If the Type is Variable , select the variable that gives the Y-axis value to the marker point.
Color	If the Type is static or Variable, select the marker point color from the color picker. If the Type is Dynamic, you can select: Manual: To select one or more colors manually. This is the default color mode associated with To specify several colors, select Customize. Global Palette: To use the global color palette that is common to all visualizations. It associat To override the automatic color mapping, select Customize. Important: The Global Palette colors are common to all visualizations in your app for consis means that to have the same color on two charts, the values must come from fields with t Use display rule: To use the color mapping defined in  Resources >  Display rules. Note: There is a limit in the number of colors available. Data Perspective Studio reuses the same color

Limit	If the Type is Dynamic , select the maximum number of marker points to display.
Marker point symbol	Select the symbol of the marker point between Circle , Empty circle , Rectangle , Round rectangle , Triangle , Diamond .
Marker point size	Select the size in pixels of the marker point.
Text position	Select where to display the text against the marker point, between top, bottom, inside, right, and left.

In the following screenshot, you can see two static marker points in a column chart. The first marker point is

